

Реализация механизма строго однократной потоковой обработки данных при работе с нереляционным хранилищем

ВВЕДЕНИЕ

Современные информационные системы всё чаще оперируют непрерывными потоками данных: журналами действий пользователей, телеметрией сервисов, событиями взаимодействия с внешними API. Объём таких потоков в крупных интернет-компаниях измеряется миллионами сообщений в секунду, а требования к задержке обработки сводятся к единицам секунд. Для построения подобных конвейеров широко применяется фреймворк Apache Spark и его подсистема структурированной потоковой обработки Spark Structured Streaming, обеспечивающая декларативное описание преобразований над неограниченными таблицами и масштабируемое исполнение на кластере.

Ключевым требованием к промышленным потоковым конвейерам является корректность данных в выходных хранилищах. На практике это означает гарантию строго однократной обработки (англ. *exactly-once*): каждое входное сообщение должно быть отражено в результирующей таблице ровно один раз вне зависимости от перезапусков, сбоев исполнителей и иных нестандартных ситуаций. Нарушение данной гарантии ведёт к появлению дубликатов или, наоборот, к утере данных, что неприемлемо для финансовой отчётности, систем расчётов с клиентами, систем обнаружения мошеннических операций и любых аналитических процессов, опирающихся на агрегированные показатели.

В качестве целевого хранилища в настоящей работе рассматривается распределённая платформа YTsaurus. Платформа объединяет в себе свойства файловой системы, документной базы данных и системы массовой аналитической обработки, причём её реляционная подсистема — динамические таблицы — поддерживает ACID-транзакции. Дополнительно YTsaurus предоставляет встроенный механизм очередей с потребителями, позволяющий описывать поток событий и его продвижение в едином формате с табличными данными. Интеграция YTsaurus с Apache Spark реализована в проекте SPYT, поэтому именно эта связка обеспечивает наиболее короткий путь от потокового источника к транзакционному хранилищу. Сочетание упомянутых свойств делает YTsaurus естественной целевой системой для исследования механизма распределённых транзакций в Spark Structured Streaming.

Текущая реализация потокового адаптера в SPYT обеспечивает только семантику доставки данных не реже одного раза (англ. *at-least-once*): данные записываются в динамическую таблицу одним вызовом, продвижение смещения потребителя выполняется отдельно после завершения записи, и при сбое между этими двумя действиями возникает повторная обработка батча. Такое поведение неприемлемо для прикладных задач, связанных с финансовыми и аналитическими расчётами, и требует разработки механизма, объединяющего запись данных и продвижение смещения в единую распределённую транзакцию.

Степень разработанности темы. Задача согласования распределённой записи с продвижением курсора потребителя освещена в литературе и промышленных решениях преимущественно применительно к связкам Apache Spark Structured Streaming с Apache Kafka и Apache Spark с реляционными хранилищами, поддерживающими двухфазный коммит. Для платформы YTsaurus, в которой курсор потребителя является элементом самой системы хранения и допускает атомарное продвижение в одной транзакции с записью данных, открытые работы и готовые программные реализации, обеспечивающие строго однократную обработку на стороне приёмника, отсутствуют. В существующем адаптере SPYT реализована только частичная схема строго однократной обработки, основанная на идемпотентной записи в сортированные динамические таблицы, ограниченная по применимости и пропускной способности (рассматривается в §1.5). Таким образом, тема исследования разработана частично, и задача общего механизма распределённой транзакционной записи остаётся открытой.

Объектом исследования является потоковая обработка данных в распределённых вычислительных системах. Предметом исследования — механизм распределённых транзакций для динамических таблиц YTsaurus в адаптере SPYT, обеспечивающий гарантию строго однократной обработки потока в Apache Spark Structured Streaming.

Цель работы — разработать и реализовать механизм поддержки распределённых транзакций для динамических таблиц YTsaurus в адаптере SPYT, обеспечивающий строго однократную семантику записи потоковых данных.

Для достижения поставленной цели сформулированы следующие задачи:

1. Проанализировать архитектуру Apache Spark Structured Streaming и текущую реализацию потокового адаптера SPYT с точки зрения семантики доставки.
2. Изучить модель транзакций YTsaurus и особенности размещения грс-прокси в архитектуре SPYT, влияющие на возможность распределённой координации.
3. Спроектировать архитектуру модуля интеграции, обеспечивающего атомарную фиксацию пары «запись данных в динамическую таблицу — продвижение смещения потребителя».
4. Реализовать спроектированный механизм в исходном коде SPYT с использованием точек расширения Apache Spark.
5. Разработать набор тестов, подтверждающих корректность семантики строго однократной обработки в условиях штатной работы и при моделировании сбоев.
6. Оценить производительность полученного решения и сравнить рассмотренные архитектурные варианты.

Научная новизна работы заключается в впервые предложенной и реализованной схеме координации таблет-транзакций YTsaurus с микропакетным циклом исполнения Spark Structured Streaming. Практическая значимость состоит в том, что разработанный модуль может быть напрямую использован в производственных потоковых конвейерах, построенных на YTsaurus и SPYT, для перехода с гарантии «не реже одного раза» к строго однократной без изменения пользовательского кода.

Методы исследования включают анализ исходного кода Apache Spark и SPYT, проектирование архитектуры на основе паттернов внедрения зависимостей через ServiceLoader и инструментации байт-кода через Java-агента, а также экспериментальную оценку реализованного механизма с использованием локального развёртывания YTsaurus.

Работа состоит из введения, пяти глав, заключения, списка использованных источников и приложения. В первой главе анализируется проблема консистентности данных в потоковой обработке и обосновывается выбор YTsaurus как целевого хранилища. Во второй главе рассматривается модель транзакций YTsaurus и особенности её реализации в архитектуре SPYT. Третья глава посвящена проектированию модуля интеграции. В четвёртой главе описывается реализация. Пятая глава содержит описание методики тестирования и результаты экспериментов. Заключение подводит итоги работы и намечает направления дальнейших исследований. В приложении А приведены листинги исходного кода ключевых компонентов реализации.

1 АНАЛИЗ ПРОБЛЕМЫ КОНСИСТЕНТНОСТИ ДАННЫХ

В настоящей главе рассмотрены гарантии доставки в системах потоковой обработки, проанализирована архитектура Apache Spark Structured Streaming, описаны основные понятия платформы YTsaugus и обоснован её выбор в качестве целевого хранилища, изучена исходная реализация потокового адаптера SPYT и формализована решаемая проблема.

1.1 Гарантии доставки в потоковой обработке

1.1.1 Семантика «не более одного раза», «не реже одного раза» и «строго однократно»

В теории распределённых систем принято выделять три уровня гарантий доставки сообщений между источником и приёмником [1, 2]. Семантика «не более одного раза» (англ. at-most-once) допускает потерю сообщений, но исключает появление дубликатов: после отправки сообщения отправитель не запрашивает подтверждения, и при сбое сообщение теряется безвозвратно. Семантика «не реже одного раза» гарантирует, что каждое сообщение будет в конечном итоге доставлено, однако допускает повторную доставку при сбоях. Семантика строго однократной обработки сочетает гарантию доставки с гарантией единственности: каждое сообщение отражается в результате ровно один раз. Именно эта семантика необходима в задачах, где входной поток описывает события с финансовым или юридическим смыслом, либо вычисляются агрегаты, чувствительные к дубликатам.

Следует подчеркнуть, что строго однократная семантика для произвольной комбинации источника и приёмника недостижима в принципе [3]. Достижимым на практике является вариант строго однократной обработки в рамках конкретной выходной системы: каждое сообщение приводит к ровно одному эффекту в выходной таблице вне зависимости от того, сколько раз потоковый движок его обработает.

1.1.2 Существующие методы решения проблемы дубликатов

Для устранения дубликатов в потоковой обработке применяются три класса методов.

Первый подход — *обеспечение идемпотентности на уровне приложения*. Пользователь дополняет выходные строки уникальным ключом и реализует в приёмнике upsert-логику: повторная запись с тем же ключом не приводит к новому эффекту. Подход требует наличия естественного ключа в данных или возможности его сгенерировать на основании смещения исходной очереди, а также поддержки upsert в целевом хранилище. К ограничениям данного подхода относятся: невозможность применения к агрегатам и иным преобразованиям, при которых выходная строка не имеет однозначного соответствия с входной, а также накладные расходы на хранение и проверку ключей. Конкретная реализация этого подхода для платформы YTsaugus и её ограничения рассматриваются в §1.5.

Второй подход — *пост-обработка дедупликации*. В выходную таблицу пишутся все строки, после чего отдельный процесс выполняет периодическую дедупликацию. К ограничениям данного подхода относятся: невозможность обеспечить корректность результата в реальном времени, удвоение объёма ввода-вывода в выходную систему и необходимость наличия признака, по которому возможна дедупликация.

Третий подход — *транзакционная фиксация*. Запись данных и фиксация прогресса чтения объединяются в одну атомарную операцию в выходной системе, что исключает рассогласование между состоянием выходной таблицы и состоянием курсора потребителя [4]. Подход не требует изменения пользовательской модели данных и не вносит дополнительных задержек на обработку. К ограничениям данного подхода относятся требования к выходной системе: она должна поддерживать транзакции, объединяющие модификацию данных и продвижение курсора.

В настоящей работе используется третий подход. Его выбор обусловлен тем, что динамические таблицы YTsaurus поддерживают полноценные ACID-транзакции, объединяющие операции записи и операции продвижения смещения потребителя очереди в одной транзакции, и не требуется вводить искусственные ключи в пользовательскую модель данных.

1.2 Архитектура Apache Spark Structured Streaming

1.2.1 Микропакетная модель исполнения

Apache Spark Structured Streaming реализует обработку неограниченного потока событий в виде последовательности микропакетов [5]. Поток рассматривается как непрерывно растущая таблица, к которой пользователь применяет реляционные преобразования. Движок периодически отрезает от потока новый фрагмент данных, выполняет над ним пользовательский план запроса и записывает результат в целевую систему. Каждый такой фрагмент имеет монотонно возрастающий идентификатор батча.

Контроль за продвижением потока возложен на класс `MicroBatchExecution`, реализующий цикл исполнения. На каждой итерации цикл выполняет следующие шаги:

1. Опрос источника на предмет нового максимального смещения.
2. Запись смещения батча в журнал смещений (`offset log`).
3. Вычисление логического плана для интервала смещений и его материализация на исполнителях (метод `runBatch`).
4. Передача результата приёмнику через `Sink.addBatch`.
5. Запись идентификатора завершённого батча в журнал коммитов (`commit log`).
6. Уведомление источников о коммите через `Source.commit`.

1.2.2 Точки расширения для источников и приёмников

Apache Spark Structured Streaming определяет интеграцию с внешними системами хранения через пару абстракций: трейт `Source`, описывающий потоковый источник данных, и трейт `Sink`, описывающий потоковый приёмник. Эти абстракции являются основными точками расширения движка: для каждой внешней системы, с которой требуется выполнять обмен данными в потоковом режиме, разрабатывается отдельная пара реализаций. Цикл исполнения `MicroBatchExecution` обращается к источнику и приёмнику исключительно через интерфейсные методы перечисленных трейтов, не делая предположений о конкретной системе хранения.

Трейт `Source` объявляет три ключевых метода. Метод получения текущего максимального смещения вызывается движком в начале каждой итерации цикла исполнения и возвращает позицию, до которой источник готов отдать данные. Метод формирования датафрейма для интервала смещений вызывается на этапе планирования батча и преобразует пару смещений «начало — конец» в распределённый набор данных, который затем материализуется на исполнителях. Метод обработки уведомления о коммите вызывается после успешной фиксации батча в журнале коммитов Spark и предоставляет источнику возможность зафиксировать продвижение состояния чтения во внешней системе.

Трейт Sink объявляет единственный метод `addBatch(batchId, data)`, в котором приёмник обязан гарантировать запись данных батча с заданным идентификатором. Соглашение Spark предусматривает, что приёмник самостоятельно отвечает за идемпотентность записи в случае повторного вызова `addBatch` с тем же идентификатором, но не диктует механизм этой идемпотентности. На приёмник возлагается также ответственность за корректное распределение записи по исполнителям и за обработку ошибок, возникающих при взаимодействии с целевой системой.

Разделение ответственности между источником и приёмником, при котором продвижение состояния чтения и собственно запись данных выполняются в разных компонентах и в разных временных фазах цикла исполнения, является принципиальным для дальнейшего анализа: именно это разделение определяет границу, на которой возникает рассогласование между записью данных и фиксацией прогресса чтения в случае сбоя.

1.2.3 Управление смещениями и контрольные точки

Журналы смещений и коммитов хранятся в каталоге контрольной точки, путь к которому задаётся пользователем. Содержимое журналов представляет собой пронумерованные файлы, каждый из которых описывает соответственно начатый или завершённый батч. Источник несёт ответственность за сериализацию и десериализацию своего смещения. После перезапуска движок Spark восстанавливает состояние из журналов: батч считается завершённым, если в журнале коммитов присутствует запись с соответствующим идентификатором; в противном случае батч переисполняется с теми же границами смещений.

Указанный механизм гарантирует семантику «не реже одного раза» на уровне Spark в предположении, что приёмник не теряет данных. Достижение строго однократной обработки требует дополнительной координации между приёмником и источником, выходящей за рамки штатных интерфейсов Spark.

1.3 Платформа YTsaurus

1.3.1 Общая архитектура и виды таблиц

YTsaurus — распределённая платформа хранения и обработки данных, развиваемая компанией Яндекс и открытая под лицензией Apache 2.0 [6]. Платформа предоставляет иерархическое пространство имён Cypress, в узлах которого размещаются файлы, документы, а также два класса таблиц: статические и динамические. Статические таблицы предназначены для пакетной аналитической обработки, поддерживают форматы строчного и колоночного хранения и оптимизированы для последовательного чтения большими блоками. Динамические таблицы являются распределённой подсистемой структурированных таблиц YTsaurus, поддерживают точечный доступ по ключу, строгую сортировку и шардирование по диапазонам, обеспечивают строгую согласованность и сериализуемую изоляцию транзакций.

1.3.2 Очереди и потребители

Поверх динамических таблиц YTsaurus предоставляет высокоуровневую абстракцию очередей и потребителей [6]. Очередь — это упорядоченная динамическая таблица, в которую события добавляются в конец и читаются в порядке добавления. Внутри очередь разбита на таблетки, каждый из которых имеет собственный монотонно возрастающий индекс строки. Потребитель описывается отдельной динамической таблицей, в строках которой для каждой пары «очередь — индекс таблетки» хранится текущее смещение чтения. Продвижение смещения выполняется специальной операцией `advanceConsumer`, доступной как в виде самостоятельного запроса, так и в составе транзакции.

Сочетание абстракций «очередь» и «потребитель» в едином хранилище принципиально важно для настоящей работы: и пользовательские табличные данные, и состояние курсора чтения находятся в одной системе, поддерживающей транзакции, что открывает возможность их атомарной модификации.

1.3.3 Обоснование выбора YTsaurus как целевой системы хранения

Выбор YTsaurus в качестве целевого хранилища обусловлен несколькими свойствами, существенными именно для задачи строго однократной потоковой обработки. Во-первых, динамические таблицы YTsaurus поддерживают полноценные ACID-транзакции, в одной транзакции допускаются как операции изменения строк (`modifyRows`), так и операции продвижения смещения потребителя (`advanceConsumer`). Это позволяет атомарно фиксировать пару «запись результата батча — продвижение курсора чтения», что является ключевым условием строго однократной обработки. Альтернативные хранилища, лишённые транзакций над курсором потребителя, требуют введения внешнего механизма координации, как правило, с использованием искусственных идемпотентных ключей.

Во-вторых, абстракции очереди и потребителя в YTsaurus реализованы поверх той же динамической таблицы, что и пользовательские данные, и используют единый механизм репликации и согласованности. Это исключает межсистемную координацию между брокером сообщений и хранилищем, типичную для связки Apache Kafka и внешнего хранилища.

В-третьих, для платформы YTsaurus уже существует адаптер SPYT, обеспечивающий глубокую интеграцию с Apache Spark: поддержку чтения и записи статических и динамических таблиц, передачу схемы, оптимизатор сортированных таблиц и базовый потоковый адаптер. Использование SPYT в качестве основы позволяет сосредоточиться на собственно проблеме распределённых транзакций, не тратя усилий на разработку соединителя с нуля.

В-четвёртых, в инфраструктуре Яндекса YTsaurus является основной платформой хранения данных, и потоковые конвейеры, доставляющие данные в динамические таблицы YTsaurus, представляют собой массовый прикладной сценарий.

1.4 Адаптер SPYT и исходная реализация потокового модуля

В настоящем разделе описывается исходное состояние потокового модуля SPYT, соответствующее ветке `branch-2.7.x` репозитория проекта, до начала работ, выполненных в рамках настоящей ВКР.

1.4.1 Состав модуля потоковой обработки

Поддержка потоковой обработки в SPYT реализована в модуле `data-source` и представлена следующими компонентами:

- `YtStreamingSource` — реализация трейта `Source`, читающая данные из очереди YTsaurus от имени заданного потребителя.
- `YtStreamingSink` — реализация трейта `Sink`, выполняющая запись в динамическую таблицу YTsaurus.
- `YtQueueOffset` — представление смещения батча, содержащее карту «индекс таблета — последний прочитанный индекс строки».
- `YtQueueRange` — Spark-партиция, описывающая полуоткрытый интервал индексов строк в одном таблете.
- `YtQueueRDD` — RDD, реализующий чтение интервала из таблета через операцию `pullConsumer`.

1.4.2 Поток данных

В методе `YtStreamingSource.latestOffset` запрашивается атрибут `queue_partitions` очереди и сравнивается с последним подтверждённым смещением, прочитанным из таблицы потребителя. На основании разности формируется новое максимальное смещение батча. В методе `YtStreamingSource.getBatch` разность смещений преобразуется в набор объектов `YtQueueRange` и оборачивается в `YtQueueRDD`. Каждая партиция RDD соответствует ровно одному tabletу очереди, в чём состоит свойство 1:1 соответствия партиций источника и tabletов очереди.

В методе `YtStreamingSink.addBatch` полученный датафрейм проходит через `foreachPartition`, в каждой партиции создаётся локальный экземпляр `YtDynamicTableWriter` и выполняется построчная запись через операцию `modifyRows` без какой-либо транзакции — каждый запрос автокоммитится клиентской библиотекой YT в момент отправки. После завершения `addBatch` Spark вызывает `YtStreamingSource.commit`, в котором единственный метод `YtQueueOffset.advance` создаёт собственную короткоживущую таблет-транзакцию, выполняет в ней `advanceConsumer` для каждого tableta и фиксирует её. Эта транзакция охватывает только продвижение смещения и не связана с записью данных, выполненной выше.

1.4.3 Семантика исходной реализации

В исходной реализации запись данных и продвижение смещения выполняются в двух полностью независимых контекстах: запись — на исполнителях через `foreachPartition` без какой-либо транзакции, продвижение — на драйвере в собственной короткоживущей таблет-транзакции, охватывающей исключительно `advanceConsumer`. Между завершением записи и завершением продвижения существует временной интервал, на котором система может выйти из строя, и атомарность пары «запись результата батча — фиксация прогресса чтения» не обеспечивается. Дополнительно, отсутствие транзакции вокруг записи означает, что частично завершённый батч оставляет в выходной таблице произвольное подмножество строк без возможности отката.

1.4.4 Сценарии нарушения консистентности

Рассмотрим сценарии сбоев, в каждом из которых исходная реализация SPYT не обеспечивает строго однократную обработку.

Сценарий А (сбой между `addBatch` и `commit`). Запись в динамическую таблицу успешно завершена, идентификатор батча зафиксирован в журнале коммитов Spark, но процесс драйвера падает до того, как `Source.commit` успел продвинуть смещение потребителя. После перезапуска Spark считает батч завершённым и не переисполняет его, однако смещение потребителя по-прежнему указывает на начало батча. Возникает рассогласование между состоянием выходной таблицы и состоянием курсора потребителя; при последующих перезапусках или удалении контрольной точки рассогласование приводит к появлению дубликатов в выходной таблице.

Сценарий Б (сбой во время `addBatch`). Часть партиций успела выполнить запись, часть — нет. Идентификатор батча в журнал коммитов не записан, поэтому Spark переисполнит батч. Данные, успевшие попасть в динамическую таблицу при первой попытке, окажутся в ней дважды.

Оба сценария устраняются объединением записи данных и продвижения смещения в одну распределённую транзакцию: при сбое транзакция отменяется целиком, а успешная фиксация одновременно делает наблюдаемыми и записанные данные, и новое смещение потребителя. Общая причина возникновения сценариев — запись данных и продвижение курсора потребителя не образуют единого атомарного действия.

1.5 Существующее частичное решение строго однократной обработки в SPYT: запись в сортированные таблицы

До начала работ, выполненных в рамках настоящей ВКР, в SPYT существовал способ получения строго однократной семантики без транзакционной координации записи и продвижения смещения. Способ основан на идемпотентной записи в сортированные динамические таблицы YTsaurus и был ранее реализован автором настоящей работы.

1.5.1 Архитектура решения

Целевая таблица создаётся как сортированная динамическая таблица, первичным ключом которой служит триплет «путь к очереди-источнику, индекс таблета очереди, индекс строки в таблете». Поля ключа автоматически проставляются приёмником в каждой выходной строке исходя из её происхождения во входной очереди: значения индексов таблета и строки переносятся из метаданных операции pullConsumer без участия пользователя. Запись в таблицу осуществляется операцией upsert: повторное появление строки с тем же ключом перезаписывает существующую строку без накопления дубликатов. Продвижение смещения потребителя выполняется в отдельной транзакции после успешной записи, как и в исходной реализации семантики «не реже одного раза», но семантическая корректность не нарушается: даже если повторно исполненный батч повторно запишет уже зафиксированные строки, их ключи совпадут с уже хранящимися и upsert не создаст дубликатов.

Схема результирующей таблицы расширяется тремя ключевыми колонками, проставляемыми приёмником автоматически (листинг 1.1).

Листинг 1.1 — Схема ключа результирующей сортированной таблицы

```
- name: __sptyt_streaming_src_queue_path
  type: string
  sort_order: ascending
- name: __sptyt_streaming_src_tablet_index
  type: int64
  sort_order: ascending
- name: __sptyt_streaming_src_row_index
  type: int64
  sort_order: ascending
# далее — пользовательские value-колонки, повторяющие схему очереди-источника
```

Тройка (queue_path, tablet_index, row_index) уникальна в пределах всей системы: путь к очереди разделяет данные нескольких источников, индекс таблета — данные разных партиций одной очереди, индекс строки — отдельные сообщения внутри партиции. Любая повторная запись строки, происходящей из одного и того же исходного сообщения, попадает на один и тот же первичный ключ и обрабатывается upsert-операцией без дублирования.

1.5.2 Ограничения существующего решения

Описанное решение не покрывает класс задач, для которого формулируется настоящая работа, в силу двух существенных ограничений.

Первое ограничение — производительность. Сортированная динамическая таблица YTsaurus поддерживает первичный ключ, по которому строки физически упорядочены, что приводит к дополнительным накладным расходам по сравнению с упорядоченной таблицей: на каждую вставку выполняется поиск точки вставки в B-дереве, при достижении компактификационных порогов запускается фоновая

компактификация диапазонов, а размер хранимого индекса растёт пропорционально объёму данных. На малых объёмах разница незаметна, однако на больших таблицах деградация записи становится существенной и продолжает расти с ростом объёма таблицы. Упорядоченная таблица, не имеющая первичного ключа, лишена этих накладных расходов и обеспечивает существенно более высокую пропускную способность записи.

Второе ограничение — несовместимость с произвольными преобразованиями. Идемпотентный ключ строится из координат входной строки в очереди и предполагает однозначное соответствие между входной и выходной строкой. При преобразованиях, нарушающих это соответствие — таких как соединения с другими источниками, оконные функции, агрегации, дедупликация по доменным признакам — выходная строка не имеет уникальных координат во входной очереди, и уникальность ключа невозможно обеспечить. В таких сценариях идемпотентная запись через сортированную таблицу принципиально неприменима, и единственным способом сохранить строго однократную обработку остаётся транзакционная координация записи и продвижения смещения.

Из перечисленных ограничений следует, что существующее частичное решение покрывает только узкий класс задач с преобразованиями без сохранения состояния и малым объёмом данных и не может быть рекомендовано как общее средство получения строго однократной обработки в SPYT. Возникает потребность в механизме, который обеспечивает строго однократную обработку для произвольных преобразований и для записи в упорядоченные таблицы YTsaurus, не уступая существующему решению по производительности.

1.6 Постановка задачи

Проведённый в предыдущих разделах анализ показал, что современная промышленная потоковая обработка данных предъявляет требование строго однократной обработки, что для интеграции Apache Spark Structured Streaming с динамическими таблицами YTsaurus реализуется только частично. Существующее в SPYT частичное решение, основанное на идемпотентной записи в сортированные таблицы, неприменимо к задачам с произвольными преобразованиями и не удовлетворяет требованию пропускной способности на больших объёмах данных. С учётом этих результатов целью настоящей работы становится обеспечение гарантии строго однократной потоковой обработки данных в Apache Spark при записи в динамические таблицы YTsaurus, не ограниченное приведёнными ранее условиями применимости.

Достижение поставленной цели требует разработки механизма, который атомарно фиксирует два действия, выполняемых в рамках обработки одного микропакета: запись результата обработки порции данных в целевую динамическую таблицу YTsaurus и сохранение прогресса чтения в систему контрольных точек Apache Spark и YTsaurus. При сбое на любом этапе обработки порции данных распределённая транзакция должна полностью откатываться, исключая появление дубликатов или потерю данных, а после успешной фиксации состояние выходной таблицы и состояние курсора потребителя должны быть согласованы между собой и с журналом коммитов Spark.

Разрабатываемый механизм должен обеспечивать гарантию строго однократной обработки при записи в упорядоченные динамические таблицы YTsaurus и сохранять корректность при произвольных преобразованиях пользовательского запроса, включая соединения, оконные функции и агрегации. По пропускной способности записи на задачах, где применимо существующее частичное решение, новый механизм не должен уступать ему. Реализация должна опираться на штатные механизмы транзакций YTsaurus и существующие в SPYT средства расширения Apache Spark — программные адаптеры и инструментацию байт-кода через Java-агент, — и не требовать изменений в пользовательском коде потокового запроса: переключение между исходной семантикой «не реже одного раза» и новой строго однократной семантикой должно выполняться установкой одного конфигурационного параметра.

2 МОДЕЛЬ ТРАНЗАКЦИЙ YTSAURUS

В настоящей главе рассмотрена транзакционная модель платформы YTsaurus: типы транзакций, их свойства и API клиентской библиотеки, а также особенности размещения грс-прокси в архитектуре SPYT, существенно влияющие на возможность реализации распределённой транзакции.

2.1 Типы транзакций YTsaurus

YTsaurus поддерживает два самостоятельных типа транзакций, различающихся областью действия и механизмом реализации [6].

2.1.1 Мастер-транзакции

Мастер-транзакции реализуются распределённой подсистемой мастер-серверов, обслуживающей метаданные Cypress. Состояние мастер-транзакции реплицируется между серверами и переживает сбой отдельных узлов. В мастер-транзакциях допускаются операции над файлами, документами и статическими таблицами Cypress, операции изменения структуры пространства имён, а также чтение динамических таблиц. Запись в динамические таблицы в мастер-транзакции не поддерживается, поскольку запись адресуется к подсистеме таблет-узлов, не участвующих в данном протоколе.

2.1.2 Таблет-транзакции

Таблет-транзакции реализуются подсистемой таблет-узлов и предназначены для работы с динамическими таблицами и очередями. В одной таблет-транзакции допускается выполнение операций `modifyRows`, `lookupRows`, `selectRows`, `advanceConsumer` над произвольным числом динамических таблиц, расположенных на разных таблет-узлах. Атомарность фиксации обеспечивается двухфазным протоколом коммита, согласованным с метаданными Cypress.

Именно таблет-транзакции являются единственным механизмом, в котором может быть атомарно объединена запись пользовательских данных в выходную динамическую таблицу и продвижение смещения потребителя очереди.

2.2 Свойства таблет-транзакций

2.2.1 ACID-гарантии

Таблет-транзакции YTsaurus обеспечивают полный набор ACID-свойств. Атомарность гарантируется двухфазным коммитом, согласованность поддерживается на уровне схем и ограничений целостности динамических таблиц, изоляция реализуется по схеме `serializable snapshot isolation`, надёжность обеспечивается репликацией `write-ahead log` на таблет-узлах [6].

2.2.2 Поддерживаемые операции

В контексте настоящей работы практически используются две операции:

- `modifyRows` — пакетная вставка, обновление и удаление строк динамической таблицы. Используется при записи результата батча в выходную таблицу.
- `advanceConsumer` — атомарное обновление смещения потребителя. Используется при фиксации завершённого батча.

Обе операции принимают идентификатор транзакции в качестве параметра либо вызываются на объекте транзакции, что обеспечивает их выполнение в рамках одного атомарного действия. Соответствующие обёртки над YT-клиентом представлены в модуле `yt-wrapper` адаптера SPYT в трейтах `YtDynTableUtils` (листинг A.9 приложения A) и `YtQueueUtils` (листинг A.10 приложения A).

2.3 Sticky-транзакции и роль грс-прокси

2.3.1 Размещение состояния транзакции

Принципиальной особенностью реализации таблет-транзакций в YTsaurus является то, что состояние транзакции (список изменений до коммита, идентификатор, таймстемпы) хранится не на таблет-узлах, а в клиентской библиотеке, исполняющейся внутри грс-прокси, через который клиент общается с кластером [6]. Таблет-узлы видят транзакцию только в момент её коммита, когда грс-прокси отправляет им сериализованный набор изменений. Такой выбор архитектуры существенно снижает нагрузку на таблет-узлы за счёт переноса буферизации мутаций на сторону прокси.

2.3.2 Параметр `sticky`

Поскольку состояние транзакции локально для одного грс-прокси, обращения к одной и той же транзакции должны направляться в один и тот же экземпляр прокси. Для этого в YTsaurus введён режим `sticky`: при создании транзакции с флагом `sticky=true` клиентская библиотека закрепляет за транзакцией конкретный грс-прокси и направляет в него все последующие запросы. В отсутствие данного флага запросы могут балансироваться между разными прокси, и любая операция над транзакцией с высокой вероятностью завершится ошибкой `No such transaction`.

В адаптере SPYT соответствующий код находится в методе `YtTransactionUtils.createTransaction` (листинг A.8 приложения A): тип транзакции выбирается равным `TransactionType.Tablet` при `sticky=true` и `TransactionType.Master` иначе.

2.3.3 Поддержание времени жизни транзакции

Таблет-транзакция имеет конечное время жизни, заданное параметром `transactionTimeout`. Чтобы транзакция не была автоматически прервана грс-прокси, клиент обязан периодически отправлять `ping`-запросы, продлевающие срок жизни. В клиентской библиотеке за это отвечает планировщик, активируемый параметрами `setPing(true)` и `setPingPeriod(...)` при создании транзакции. В коде SPYT данная конфигурация задаётся в методе `YtTransactionUtils.createTransaction` (листинг A.8 приложения A), а функция `YtTransactionUtils.pingTransaction` (там же) выполняет дополнительные синхронные `ping`-запросы в фоновом потоке.

2.4 Архитектура грс-прокси в SPYT

Стандартная схема развёртывания YTsaurus предполагает выделенный пул грс-прокси, общий для всех клиентов кластера. В этой схеме два процесса клиента, обращающиеся к кластеру, могут как попасть на один и тот же прокси, так и оказаться обслуживаемыми разными прокси, в зависимости от балансировщика.

Адаптер SPYT работает в специальной среде исполнения Apache Spark поверх YTsaurus, и в этой среде существует две конфигурации размещения грс-прокси.

2.4.1 Конфигурация с грс-прокси в job-проху

В типичной для SPYT конфигурации грс-прокси встраивается непосредственно в job-проху — процесс-обёртку, в котором YTsaurus запускает каждый компонент Spark (драйвер и каждый исполнитель). Такая схема обеспечивает локальность сетевых обращений и снижает нагрузку на общий пул прокси кластера. Однако следствием встраивания является то, что у каждого процесса Spark — собственный изолированный экземпляр грс-прокси, и состояние таблет-транзакции, созданной в драйвере, физически не доступно из исполнителей.

Поведение управляется конфигурационным параметром `spark.ytsaurus.grs.job.proxy.enabled`, по умолчанию равным `true`. При значении `false` все компоненты Spark обращаются к общему пулу грс-прокси кластера.

2.4.2 Изоляция транзакций между процессами

В конфигурации с грс-прокси в job-проху попытка передать идентификатор таблет-транзакции, созданной в драйвере, на исполнитель приводит к ошибке `No such transaction`. Эта ошибка указывает на то, что транзакция существует в одном грс-прокси, тогда как обращение к ней пришло в другой грс-прокси, который о её существовании не знает.

Изоляция между процессами устраняется одним из двух способов. Первый — сведение всех обращений к транзакции в один процесс Spark: тогда вся работа с транзакцией происходит через единственный грс-прокси, встроенный в job-проху этого процесса, и проблема видимости не возникает. Второй — переход на общий пул грс-прокси (`spark.ytsaurus.grs.job.proxy.enabled=false`) при условии, что транзакция создана в режиме `sticky` и клиентская библиотека на исполнителе настроена на использование того же прокси, что и драйвер; это требует расширения клиентской библиотеки YT функцией присоединения к существующей транзакции. Оба способа подробно рассматриваются в главе 3.

2.5 API клиентской библиотеки

2.5.1 Класс ApiServiceTransaction

Основным объектом, представляющим таблет-транзакцию в Java- и Scala-коде, является `ApiServiceTransaction`. Объект инкапсулирует идентификатор транзакции, ссылку на грс-прокси и предоставляет методы `modifyRows`, `advanceConsumer`, `commit`, `abort`, `ping`. Идентификатор транзакции доступен через метод `getId` и пригоден для логирования и передачи в качестве параметра в операции, поддерживающие `transactionId`.

2.5.2 Утилиты SPYT

В модуле `yt-wrapper` адаптера SPYT агрегированы обёртки над клиентской библиотекой:

- `YtTransactionUtils` (листинг A.8 приложения A) — создание, коммит, отмена и пинг транзакций.
- `YtDynTableUtils` (листинг A.9 приложения A) — операции над динамическими таблицами, включая `insertRows` с опциональным идентификатором родительской транзакции.
- `YtQueueUtils` (листинг A.10 приложения A) — операции над очередями и потребителями. Содержит две перегрузки метода `advanceConsumer`: принимающую объект `ApiServiceTransaction` (вызов выполняется на самом объекте транзакции) и принимающую идентификатор родительской транзакции

в виде строки (вызов выполняется на клиенте с указанием `transactionId` в запросе).

Наличие двух перегрузок отражает фундаментальное различие между сценарием «транзакция создана локально, объект доступен» и сценарием «транзакция создана на другом узле, локально известен только идентификатор». Первый сценарий пригоден для одного процесса, второй — для распределённой координации.

2.6 Ограничения и следствия для распределённой записи

Сведём наблюдения главы в следующие выводы.

1. Запись в выходную динамическую таблицу `YTsauros` и продвижение смещения потребителя могут быть атомарно объединены только в одной таблет-транзакции.
2. Состояние таблет-транзакции локально для конкретного грс-прокси и не реплицируется между прокси.
3. В стандартной для `SPYT` конфигурации с грс-прокси, встроенным в `job-проху`, разные процессы `Spark` обслуживаются разными грс-прокси, и совместная работа с одной таблет-транзакцией возможна одним из двух способов: сведением всех обращений к транзакции в один процесс `Spark` либо переходом на общий пул грс-прокси с дополнительной поддержкой клиентской библиотеки — операцией присоединения к транзакции по её идентификатору и адресу прокси.

Перечисленные ограничения формируют пространство допустимых архитектурных решений, которое исследуется в главе 3.

3 ПРОЕКТИРОВАНИЕ МОДУЛЯ ИНТЕГРАЦИИ

В настоящей главе формулируются требования к модулю распределённой транзакции, описываются используемые точки расширения `Apache Spark` и `SPYT`, анализируется соответствие партиций потокового источника и приёмника, рассматриваются два архитектурных решения и обосновывается выбор одного из них.

3.1 Требования к модулю

3.1.1 Функциональные требования

К модулю предъявляются следующие функциональные требования.

1. Гарантия строго однократной обработки потоковых данных при записи из очереди `YTsauros` в динамическую таблицу `YTsauros` через `Spark Structured Streaming`.
2. Атомарность пары «запись батча в выходную таблицу — продвижение смещения потребителя на конец батча» в рамках единой таблет-транзакции `YTsauros`.
3. Корректное поведение при сбоях: либо обе операции зафиксированы, либо обе откатены, и переисполнение батча не приводит к появлению дубликатов и потере данных.
4. Прозрачность для пользовательского кода: переключение между семантиками «не реже одного раза» и строго однократной выполняется конфигурационным

параметром `spark.yt.streaming.transactional`, без модификации описания потокового запроса.

3.1.2 Нефункциональные требования

1. Совместимость с Apache Spark 3.5.x как с приоритетной поддерживаемой версией адаптера SPYT, при сохранении совместимости с Apache Spark 3.2.x.
2. Минимизация накладных расходов на батч в установившемся режиме работы.
3. Использование существующих в SPYT механизмов расширения Apache Spark — адаптеров и инструментации байт-кода через Java-агент.
4. Совместимость с существующими реализациями `YtStreamingSource` и `YtStreamingSink`: отсутствие необходимости перепроектирования базовых компонентов.

3.2 Точки расширения Apache Spark и SPYT

3.2.1 ServiceLoader и SPI

Часть точек расширения, доступных в API модуля интеграции, реализуется средствами стандартного механизма Java `ServiceLoader`. Реализация интерфейса регистрируется в файле `META-INF/services/`, и Spark или SPYT загружает её в момент инициализации. В реализации одного из решений, рассматриваемых в §3.4, используется интерфейс `StreamingTransactionSupport`, объявляющий контракт жизненного цикла распределённой транзакции, и его конкретная реализация `YtsaurusStreamingTransactionSupport`, регистрируемая через `ServiceLoader`.

3.2.2 Декорирование внутренних методов Spark

Часть необходимой функциональности располагается во внутренних классах Spark, не предусматривающих штатного расширения. Для воздействия на эти классы в SPYT применяется механизм инструментации байт-кода, реализованный в модуле `spark-patch`. Главный компонент модуля — Java-агент `SparkPatchAgent`, регистрируемый через параметр `-javaagent` JVM драйвера и исполнителей. Агент сканирует `classpath` в поиске классов-декораторов, помеченных аннотациями `@PatchSource` и `@Decorate`, и подменяет реализации соответствующих методов в исходных классах Spark, сохраняя оригинальное поведение и оборачивая его в пользовательский код.

В реализации одного из решений, рассматриваемых в §3.4, через данный механизм декорируется метод `runBatch` класса `MicroBatchExecution` с целью внедрения логики управления контекстом батча между вызовами `Source.getBatch`, `Sink.addBatch` и `Source.commit`.

3.2.3 ThreadLocal-контекст

Третья точка взаимодействия — `ThreadLocal`-переменные, используемые для передачи данных между методами одного потока без изменения сигнатур. В реализации одного из решений, рассматриваемых в §3.4, используется контекст `YtStreamingTransactionContext`, хранящий идентификатор родительской транзакции и адрес грс-прокси, на которой была создана транзакция. Использование `ThreadLocal` оправдано тем, что обработка одного батча выполняется на драйвере в одном потоке, и контракт `MicroBatchExecution` гарантирует последовательное исполнение.

3.3 Соответствие партиций источника и приёмника

3.3.1 Соответствие 1:1 на уровне источника

В реализации источника соответствие между партициями RDD и таблетами очереди установлено явно. В классе `YtQueueRange` индекс Spark-партиции переопределён равным индексу таблета:

```
case class YtQueueRange(tabletIndex: Int, lowerIndex: Long,
                        upperIndex: Long) extends Partition {
  override def index: Int = tabletIndex
}
```

В методе `YtQueueRDD.getPartitions` партиции возвращаются отсортированными по `tabletIndex`, что обеспечивает стабильный порядок и взаимно однозначное соответствие между партициями RDD и таблетами очереди.

Свойство 1:1 соответствия позволяет в принципе организовать на каждом исполнителе независимую таблет-транзакцию, обрабатывающую один таблет: запись в выходную таблицу строк одного таблета и продвижение смещения потребителя для этого же таблета.

3.3.2 Потеря соответствия после преобразований

Однако данное свойство справедливо лишь до тех пор, пока RDD источника проходит через узкие преобразования (`map`, `filter`, `mapPartitions`), сохраняющие партиционирование. Любое широкое преобразование пользовательского пайплайна — `groupBy`, `join`, `repartition`, оконная агрегация — выполняет `shuffle` и пересоздаёт партиционирование результата. После `shuffle` партиция приёмника содержит строки, происходящие из произвольного подмножества таблеток источника, и независимое продвижение смещения по таблету становится невозможным: исполнитель не знает, какие таблетки внесли вклад в его партицию и в каком объёме.

Между источником и приёмником ничто не гарантирует отсутствие широких преобразований, поэтому решение, рассчитанное на сохранение партиционирования, нельзя считать применимым в общем случае.

3.3.3 Следствия для архитектуры

Из анализа следует, что распределённая координация транзакции должна допускать одно из двух:

- сведение всего батча к одному исполнителю, способному провести транзакцию целиком;
- предоставление множеству исполнителей возможности участвовать в одной общей транзакции, координируемой драйвером.

Эти два способа порождают два архитектурных варианта, рассмотренных далее.

3.4 Архитектурные решения

В §3.3.3 показано, что распределённая координация транзакции допускает либо сведение всего батча к одному исполнителю, либо вовлечение множества исполнителей в одну общую транзакцию, координируемую драйвером. Этим двум стратегиям соответствуют два архитектурных решения, рассмотренных ниже.

3.4.1 Решение 1: запись на одном исполнителе

Идея решения состоит в том, чтобы перед записью применять к датафрейму операцию `coalesce(1)`. После её применения весь батч обрабатывается одним заданием Spark на одном исполнителе. Этот исполнитель самостоятельно открывает таблет-транзакцию через локальный грс-прокси, встроенный в его `job-proxy`, выполняет в этой транзакции запись данных батча и операцию `advanceConsumer` для всех таблеток очереди, после чего фиксирует транзакцию. На драйвере метод `Source.commit` при включённом флаге `spark.yt.streaming.transactional` пропускает действия по продвижению смещения, поскольку оно уже выполнено в составе фиксированной транзакции.

Жизненный цикл батча в решении 1 выглядит следующим образом.

1. На драйвере метод `Source.getBatch` формирует RDD из таблеток очереди.
2. На драйвере метод `Sink.addBatch` применяет к датафрейму `coalesce(1)` и запускает задание `foreachPartition` с одной партицией.
3. На выбранном исполнителе открывается таблет-транзакция через локальный грс-прокси.
4. В этой транзакции выполняется построчная запись данных батча через `modifyRows`.
5. В этой же транзакции для каждого таблетки очереди, попавшего в батч, выполняется операция `advanceConsumer` с соответствующим конечным смещением.
6. Транзакция фиксируется методом `commit`. При сбое на любом из шагов 4–6 транзакция отменяется методом `abort`, и Spark переисполняет батч.
7. На драйвере метод `Source.commit` пропускает действия по продвижению смещения.

С точки зрения реализации решение 1 локально для модуля `data-source-extended`: достаточно ввести альтернативную ветку в методе `YtStreamingSink.addBatch`, отрабатывающую при включённом флаге `spark.yt.streaming.transactional`, и зеркальную ветку в `YtStreamingSource.commit`, отключающую штатное продвижение смещения. Транзакция рождается, живёт и умирает внутри замыкания, переданного в `foreachPartition`, и не пересекает границу JVM. По этой причине решение 1 не требует ни инструментации байт-кода `MicroBatchExecution`, ни введения интерфейса `StreamingTransactionSupport`, ни `ThreadLocal`-контекстов передачи идентификатора транзакции между фазами обработки батча.

Сильная сторона решения — простота реализации, локализация изменений в одном модуле SPYT и отсутствие изменений в сторонних проектах. Решение опирается на штатный механизм таблет-транзакций, описанный в главе 2, и не требует расширения клиентской библиотеки YT.

Слабая сторона решения — потеря параллелизма записи. Операция `coalesce(1)` сводит все партиции к одной без `shuffle`, и после её применения исходный план превращается в одну стадию из одного таска: пользовательские преобразования, чтение из таблеток очереди, формирование выходных строк, сериализация в формат `modifyRows` и сетевая передача в грс-прокси выполняются в одном потоке одного процесса. На пути к динамической таблице возникают два узких места: пропускная способность одного исполнителя и пропускная способность одного грс-прокси. Совместный потолок определяется минимумом двух, и обычно лимитирует именно исполнитель: данные не успевают сформироваться в темпе, при котором прокси становится узким местом. Дополнительно, накопление всего батча в памяти одного процесса повышает риск его исчерпания на больших батчах.

3.4.2 Решение 2: общий пул грс-прокси и `AttachTransaction`

Идея решения состоит в том, чтобы изменить конфигурацию SPYT на общий пул грс-прокси (флагом `spark.ytsaurus.rpc.job.proxy.enabled=false`) и расширить клиентскую библиотеку YT операцией присоединения исполнителя к существующей таблет-транзакции, открытой драйвером. На стороне библиотеки реализуется функция `attachTransaction(transactionId, proxyAddress)`,

создающая локальный объект `ApiServiceTransaction`, который направляет запросы в указанный грс-прокси с указанным идентификатором. Поскольку все исполнители работают с тем же грс-прокси, что и драйвер, состояние таблёт-транзакции остаётся видимым каждому из них (см. §2.4).

Жизненный цикл батча в решении 2 выглядит следующим образом (визуализация на уровне последовательности вызовов между драйвером, исполнителями и грс-прокси приведена в §3.5.4).

1. На драйвере метод `Source.getBatch` формирует RDD из таблетов очереди.
2. На драйвере непосредственно перед `Sink.addBatch` открывается таблёт-транзакция; её идентификатор и адрес грс-прокси, на которой она открыта, сохраняются в `YtStreamingTransactionContext`.
3. Spark материализует RDD согласно пользовательскому плану и запускает `foreachPartition` со штатным числом партиций.
4. На каждом исполнителе через `attachTransaction` создаётся локальный объект транзакции, в котором выполняется запись данных партиции операцией `modifyRows`.
5. После завершения всех заданий записи драйвер выполняет `advanceConsumer` для каждого таблета очереди в той же транзакции.
6. Драйвер фиксирует транзакцию методом `commit`. При сбое любой задачи записи или последующих операций драйвер вызывает `abort`, и Spark переисполняет батч.
7. На драйвере метод `Source.commit` пропускает действия по продвижению смещения.

С точки зрения реализации решение 2 требует трёх компонентов, рассмотренных в §3.2: интерфейса `StreamingTransactionSupport`, регистрируемого через `ServiceLoader`, инструментации метода `MicroBatchExecution.runBatch` через Java-агент и пары `ThreadLocal`-контекстов для передачи идентификатора транзакции и параметров батча между методами драйвера. Дополнительно требуется расширение клиентской библиотеки YT функцией `attachTransaction`, что затрагивает соседний проект.

Преимуществом данного решения является сохранение параллелизма записи. Каждый исполнитель обрабатывает свою партицию на своих ядрах, в своей JVM, и сериализация выходных строк, формирование запросов `modifyRows` и заполнение сетевого канала к грс-прокси выполняются параллельно на всех исполнителях. На пути к динамической таблице остаётся одно узкое место — общая грс-прокси, через которую проходят все запросы транзакции, — но оно является серверным компонентом и при равных ресурсах кратно превосходит пропускную способность одного исполнителя. Дополнительно, объём записи в рамках одной транзакции регулируется параметром `spark.yt.write.dynBatchSize`, который определяет количество строк, накапливаемых перед отправкой запроса `modifyRows`. Это позволяет балансировать между количеством запросов к грс-прокси и размером каждого запроса, адаптируясь под характеристики сети и производительность прокси.

Слабая сторона решения — повышенная сложность реализации и эксплуатации. Использование инструментации байт-кода Spark делает код чувствительным к смене минорных версий Spark и усложняет отладку: сбой в декорированном методе наблюдается как ошибка во внутреннем коде Spark, не имеющая прямого соответствия в исходных текстах SPYT. Расширение клиентской библиотеки YT функцией `attachTransaction` распространяет область изменений за пределы SPYT.

3.4.3 Сравнение решений

В таблице 3.1 приводится сравнение существующего частичного решения (§1.5), решения 1 и решения 2 по характеристикам, существенным для постановки задачи (§1.6).

Таблица 3.1 — Сравнение существующего и предлагаемых решений

Характеристика	Существующее решение (§1.5)	Решение 1	Решение 2
Целевой тип таблицы	сортированная динамическая	упорядоченная динамическая	упорядоченная динамическая
Семантика при произвольных преобразованиях	«не реже одного раза»	строго однократная	строго однократная
Семантика при stateless-преобразованиях с малым объемом таблицы	строго однократная	строго однократная	строго однократная
Параллелизм записи	сохраняется	теряется	сохраняется
Узкие места записи	первичный ключ	один исполнитель и общая грс-прокси	общая грс-прокси
Зависимость пропускной способности от объема таблицы	деградация по мере роста	отсутствует	отсутствует
Зависимость пропускной способности от размера батча	линейная	ООМ-риск на больших батчах	линейная
Изменения в SPYТ	нет	локальные в data-source-extended	в data-source-extended, spark-patch, spark-adapter
Изменения в сторонних проектах	нет	нет	расширение клиентской библиотеки YT
Использование инструментации байт-кода Spark	нет	нет	да

3.4.4 Обоснование выбора решения 2

Из таблицы 3.1 следует, что по постановке задачи (§1.6) ни существующее решение, ни решение 1 не удовлетворяют всем требованиям одновременно.

Существующее решение исключается двумя свойствами, явно противоречащими §1.6: оно ограничивает область применимости преобразованиями, не нарушающими однозначное соответствие между входной и выходной строкой, и его пропускная способность деградирует с ростом объема целевой таблицы.

Решение 1 удовлетворяет требованиям корректности и применимости, но не удовлетворяет требованию по пропускной способности. По всем характеристикам производительности оно строго уступает решению 2: к ограничению со стороны общей грс-прокси, наследуемому обоими решениями, добавляется ограничение со стороны одного исполнителя, через который сериализуется запись всего батча. Преимущество решения 1 — простота реализации — окупается однократно, тогда как ограничение пропускной способности проявляется на каждом батче эксплуатации.

Решение 2 удовлетворяет всем требованиям §1.6: обеспечивает строго однократную обработку при произвольных преобразованиях, не привязано к специфическому типу таблицы и сохраняет параллелизм записи, не уступая существующему решению по пропускной способности на задачах, где последнее применимо.

Стоимость реализации, выраженная в инструментации байт-кода Spark и расширении клиентской библиотеки YT, является разовой и не влияет на эксплуатационные характеристики.

На основании изложенного к реализации принимается решение 2. Решение 1 в дальнейших главах работы не рассматривается, кроме контекстов, где оно явно полезно как точка сравнения.

3.5 Архитектура выбранного решения

В настоящем разделе фиксируется состав компонентов решения 2, распределение ответственности между ними, перечень изменений в существующих частях SPYT и поток управления при обработке одного микропакета. Формальное описание протокола атомарной фиксации, включая разбор поведения при сбоях, вынесено в §3.6.

3.5.1 Состав модулей и распределение ответственности

Реализация решения 2 затрагивает четыре модуля Gradle-проекта SPYT и одну точку расширения клиентской библиотеки YT. Распределение ответственности между ними соответствует разделению функциональности на устойчивые и изменяемые относительно версии Spark компоненты.

Модуль `data-source-extended` содержит существующие классы потокового источника `YtStreamingSource`, потокового приёмника `YtStreamingSink` и поддерживающие их `YtQueueOffset`, `YtQueueRange`, `YtQueueRDD`. В рамках решения 2 в этот модуль вносится альтернативная ветка обработки батча, активируемая флагом `spark.yt.streaming.transactional`, и реализация SPI `YtsaurusStreamingTransactionSupport`, инкапсулирующая жизненный цикл таблет-транзакции на драйвере.

Модуль `spark-adapter` содержит интерфейс `StreamingTransactionSupport` и пару `ThreadLocal`-контекстов, передающих идентификатор транзакции и параметры батча между фазами обработки на драйвере. Размещение интерфейса в `spark-adapter`, а не в `data-source-extended`, продиктовано необходимостью обращаться к нему из декоратора `MicroBatchExecution`, расположенного в `spark-patch`.

Модуль `spark-patch` содержит декоратор метода `MicroBatchExecution.runBatch`, помеченный аннотациями `@PatchSource` и `@Decorate` (см. §3.2.2). Декоратор оборачивает оригинальный метод вызовами начальной и завершающей фаз, делегируемые реализации SPI, загруженной через `ServiceLoader`.

Модуль `yt-wrapper` содержит вспомогательные утилиты для работы с таблет-транзакциями `Ytsaurus`. В нём вводится утилита, оборачивающая операцию `attachTransaction`, расширяющая клиентскую библиотеку YT.

Точкой расширения клиентской библиотеки YT является функция `attachTransaction(transactionId, proxyAddress)`, создающая локальный объект `ApiServiceTransaction`, направляющий запросы в указанный грс-прокси с указанным идентификатором (см. §2.4.2, второй путь устранения изоляции).

3.5.2 Компонентная диаграмма

На рисунке 3.1 показан состав компонентов решения 2 и связи между ними. Прямоугольниками со скруглёнными углами обозначены процессы JVM (драйвер, исполнитель, грс-прокси), сплошными прямоугольниками — модули и классы внутри процессов, стрелками — направление вызовов.

flowchart TB
subgraph Driver[JVM драйвера]

```

        MBE[MicroBatchExecution.runBatch]
        DEC[Декоратор runBatch<br/>spark-patch]
        SPI[YTsaurusStreamingTransactionSupport<br/>data-source-
extended]
        SRC[YtStreamingSource]
        SINK[YtStreamingSink]
        TLC[YtStreamingTransactionContext<br/>YtStreamingBatchContext]
        QUO[YtQueueOffset.advance...]
    end
    subgraph Executor[JVM исполнителя]
        EXR[foreachPartition closure]
        ATT[attachTransaction<br/>yt-wrapper]
        MOD[modifyRows]
    end
    subgraph Proxy[rpc-прокси общего пула]
        TX[(таблет-транзакция)]
    end
    end
    DEC --> MBE
    DEC --> SPI
    SPI --> TLC
    MBE --> SRC
    MBE --> SINK
    SRC --> TLC
    SINK --> TLC
    SINK --> EXR
    SPI --> QUO
    SPI --> TX
    QUO --> TX
    EXR --> ATT
    ATT --> MOD
    MOD --> TX

```

Рисунок 3.1 — Компонентная диаграмма решения 2

Драйвер сосредотачивает управление жизненным циклом транзакции в реализации SPI `YTsaurusStreamingTransactionSupport`, вызываемой из декоратора `runBatch`. Передача идентификатора транзакции и параметров батча между методами драйвера происходит через `ThreadLocal`-контексты. Исполнители не открывают и не фиксируют транзакцию: они лишь присоединяются к ней через `attachTransaction` и направляют запросы `modifyRows` в тот же `rpc`-прокси, на котором она открыта.

3.5.3 Изменения в существующих компонентах SPYT

Перечень компонентов, затрагиваемых реализацией решения 2, и характер изменений приведён ниже.

1. `YtStreamingSource` (листинг А.5) — метод `getBatch` не изменяется; в методе `commit` при наличии активного контекста транзакции смещение потребителя продвигается в составе этой транзакции через `offsetProvider.advance(..., parentTransactionId)`; при отсутствии контекста и включённом флаге `spark.yt.streaming.transactional` метод завершается без действий (транзакция ещё не зафиксирована декоратором).
2. `YtStreamingSink` (листинг А.6) — в методе `addBatch` идентификатор открытой драйвером транзакции и адрес `rpc`-прокси извлекаются из `YtStreamingTransactionContext` и доставляются на исполнители широковещательными переменными; в замыкании `foreachPartition` создаётся локальный объект транзакции через `attachTransaction`, и запись `modifyRows` выполняется в этой транзакции вместо открытия собственной.
3. `YtDynamicTableWriter` (листинг А.7) — расширяется необязательным параметром `parentTransaction: Option[ApiServiceTransaction]`; при его

- наличии запросы `modifyRows` направляются в переданную транзакцию, а не открывают собственную.
4. `MicroBatchExecution.runBatch` (Apache Spark) — декорируется через механизм `spark-patch`. Декоратор перед вызовом оригинального метода открывает таблет-транзакцию и помещает её идентификатор и адрес прокси в `YtStreamingTransactionContext`; после возврата из оригинального метода вызывает `commitOffsets`, который через `Source.commit` продвигает смещение потребителя в составе той же транзакции, после чего фиксирует транзакцию; при исключении — отменяет транзакцию и откатывает запись в `commit-лог` Spark. Оригинальное поведение `runBatch` сохраняется без изменений.
 5. Новый класс `YtSaurusStreamingTransactionSupport` (модуль `data-source-extended`) — реализация `SPI StreamingTransactionSupport`, регистрируемая в `META-INF/services/`. Инкапсулирует создание таблет-транзакции (`TransactionType.Tablet, sticky=true`), получение адреса grpc-прокси через рефлекссию, а также операции `commit` и `abort`.
 6. Новый интерфейс `StreamingTransactionSupport` (модуль `spark-adapter`) — контракт жизненного цикла распределённой транзакции с методами `isTransactionalStreamingEnabled`, `createTransaction`, `setTransaction`, `clearTransactionId`, `isRecoveryNeeded`, `markRecoveryNeeded`, `clearRecoveryNeeded`.
 7. Новый `ThreadLocal`-контекст `YtStreamingTransactionContext` (модуль `data-source-extended`) — хранит `StreamingTxContext(txId, stickyAddress)` и флаг `recoveryNeededFlag`.
 8. Расширение `yt-wrapper` функцией `attachTransaction(transactionId)` — оборачивает операцию `AttachTransaction` клиентской библиотеки YT.

3.5.4 Поток управления при обработке батча

На рисунке 3.2 показан поток управления при штатной обработке одного микропакета в решении 2. Указаны только сообщения, относящиеся к управлению транзакцией; пользовательская логика и стандартные сообщения Spark опущены.

```
sequenceDiagram
    participant DEC as Декоратор runBatch
    participant SPI as YtSaurusStreamingTransactionSupport
    participant SRC as YtStreamingSource
    participant SINK as YtStreamingSink
    participant EX as Исполнитель
    participant PX as grpc-прокси
    DEC->>SPI: createTransaction
    SPI->>PX: startTransaction (TransactionType.Tablet)
    PX-->>SPI: transactionId, proxyAddress
    SPI->>SPI: setContext(txId, proxyAddress) in ThreadLocal
    DEC->>SRC: getBatch
    SRC-->>DEC: DataFrame
    DEC->>SINK: addBatch
    SINK->>SINK: read txId, proxyAddress from ThreadLocal
    SINK->>EX: foreachPartition (broadcast txId, proxyAddress)
    EX->>PX: attachTransaction(txId)
    EX->>PX: modifyRows on attached transaction
    EX-->>SINK: success
    DEC->>DEC: commitOffsets(availableOffsets)
    DEC->>SRC: commit(endOffset)
    SRC->>PX: advanceConsumer in transaction (txId)
    PX-->>SRC: ok
    SRC-->>DEC: ok
    DEC->>SPI: commit()
    SPI->>PX: commit transaction
    PX-->>SPI: ok
```

Рисунок 3.2 — Поток управления при штатной обработке батча в решении 2

Декоратор `runBatch` получает управление перед оригинальным методом и вызывает `createTransaction SPI`, которая открывает таблет-транзакцию (`TransactionType.Tablet`) через `grpc-прокси` общего пула и сохраняет её идентификатор и адрес прокси в `YtStreamingTransactionContext`. Оригинальный `runBatch` выполняется без изменений: вызывается `Source.getBatch`, формируется план, выполняется `Sink.addBatch`. На стороне приёмника идентификатор транзакции и адрес `grpc-прокси` извлекаются из `YtStreamingTransactionContext` и доставляются на исполнители широковещательными переменными; в замыкании `foreachPartition` создаётся локальный объект транзакции через `attachTransaction`, и запись `modifyRows` выполняется в нём. После возврата из оригинального `runBatch` декоратор вызывает `commitOffsets`, который обходит все источники и вызывает `Source.commit(endOffset)`. В методе `YtStreamingSource.commit` при наличии активного контекста транзакции смещение потребителя продвигается в составе той же транзакции через `advanceConsumer`. Затем декоратор фиксирует транзакцию вызовом `commit()` на объекте `StreamingTransactionHandle`.

3.6 Протокол атомарной фиксации батча

В настоящем разделе формализован протокол атомарной фиксации батча в решении 2. Протокол описывает последовательность шагов, инварианты после каждого шага и поведение при сбоях.

3.6.1 Последовательность шагов

Протокол состоит из шести шагов, выполняемых в рамках одного батча. На первом шаге драйвер создаёт таблет-транзакцию на `grpc-прокси` с параметром `sticky=true`. Транзакция получает уникальный идентификатор `txId`, а адрес прокси сохраняется как `stickyAddress`. Контекст, состоящий из пары (`txId`, `stickyAddress`), устанавливается в `YtStreamingTransactionContext`. После этого транзакция открыта и активна на одном `grpc-прокси`, а контекст доступен в текущем потоке драйвера.

На втором шаге Spark вызывает метод `Source.getBatch`, который формирует RDD из таблеток очереди. Чтение данных выполняется без транзакции, поскольку чтение не требует атомарности. К этому моменту данные батча загружены в RDD, а транзакция остаётся активной.

Третий шаг заключается в широковещании контекста. Драйвер упаковывает `txId` и `stickyAddress` в широковещательные переменные и передаёт их на исполнители. После этого контекст транзакции доступен на всех исполнителях, а транзакция продолжает быть активной.

На четвёртом шаге происходит запись данных на исполнителях. Каждый исполнитель создаёт клиента, зафиксированного на `stickyAddress`, и присоединяется к транзакции через вызов `attachTransaction(txId)`. В рамках присоединённой транзакции выполняются запросы `modifyRows` для вставки строк в выходную таблицу. После завершения этого шага все строки батча записаны в транзакцию, но транзакция ещё не закоммичена, поэтому записи невидимы другим транзакциям.

Пятый шаг — продвижение смещения потребителя. Драйвер вызывает метод `Source.commit`, внутри которого `YtStreamingSource.commit` выполняет операцию `advanceConsumer` в составе той же транзакции, используя `txId` из контекста. В результате смещение потребителя продвинуто в транзакции, а запись данных и продвижение смещения находятся в одной транзакции.

На шестом шаге драйвер вызывает метод `commit()` на транзакции. После успешной фиксации данные становятся видимыми, а смещение потребителя сохраняется. Батч зафиксирован атомарно, данные записаны и видны, а смещение потребителя продвинуто.

3.6.2 Поведение при сбоях

При сбое на первом шаге, то есть при открытии транзакции, исключение передаётся в декоратор, который не устанавливает контекст и делегирует обработку штатному механизму Spark. Батч переисполняется. В этом случае транзакция не создана, смещение не продвинуто, а данные не записаны.

Если сбой происходит на четвёртом шаге при записи на исполнителе, исключение из `foreachPartition` передаётся в драйвер. Декоратор отменяет транзакцию через вызов `abort()`, устанавливает флаг `recoveryNeeded` и откатывает запись в `commit`-лог Spark через `commitLog.purgeAfter(batchId - 1)`. Транзакция отменена, поэтому записи в таблице невидимы. `Commit`-лог откатлен, поэтому Spark переисполнит батч с теми же смещениями.

При сбое на пятом шаге, то есть при выполнении `advanceConsumer`, исключение из `Source.commit` перехватывается декоратором, который отменяет транзакцию и откатывает `commit`-лог. Транзакция отменена, поэтому записи в таблице невидимы, смещение не продвинуто, а Spark переисполнит батч.

Если сбой происходит на шестом шаге при фиксации транзакции, исключение из `commit()` перехватывается декоратором, который устанавливает флаг `recoveryNeeded`. Транзакция может находиться в неопределённом состоянии, но `YTsauros` гарантирует её завершение — либо `commit`, либо `abort`. При повторном батче декоратор проверит флаг и синхронизирует состояние. Если транзакция закоммичена, то данные видны и смещение продвинуто. Если транзакция отменена, то данные невидимы и смещение не продвинуто. В обоих случаях консистентность сохраняется.

При потере связи с `грс`-прокси исполнитель наблюдает сетевую ошибку при выполнении `modifyRows`. Исключение передаётся в драйвер, который отменяет транзакцию. При переисполнении батча новая транзакция открывается на доступном прокси. Отменённая транзакция не оставляет видимых записей, а переисполнение батча читает те же данные.

3.6.3 Взаимодействие с `commit`-логом Spark

Spark Structured Streaming сохраняет смещения в `commit`-лог после успешного выполнения батча. В транзакционном режиме декоратор контролирует эту логику. При штатном завершении декоратор вызывает `commitOffsets()`, который обновляет `commit`-лог, после чего фиксирует транзакцию. Смещение в `commit`-логе соответствует продвинутому смещению потребителя в `YTsauros`.

Если сбой происходит после записи в `commit`-лог, декоратор проверяет флаг `commitLogWritten` и при необходимости откатывает запись через `commitLog.purgeAfter(batchId - 1)`, заставляя Spark переисполнить батч. При сбое до записи в `commit`-лог сам `commit`-лог не изменяется, и Spark переисполняет батч с теми же смещениями.

При восстановлении после сбоя, например при рестарте драйвера, Spark читает смещения из `commit`-лога, но `YtStreamingSource.getBatch` приоритизирует состояние потребителя `YTsauros` как источник истины. Если потребитель не был продвинут, то есть транзакция не была закоммичена, батч переисполняется с корректными смещениями.

3.7 Риски и компромиссы

В настоящем разделе перечислены риски, существенные для эксплуатации решения 2, и принятые архитектурные компромиссы. Под риском понимается событие или свойство среды, способное привести к деградации функциональных или

нефункциональных требований §3.1; под компромиссом — осознанное архитектурное решение, ограничивающее одно из требований в обмен на удовлетворение другого.

3.7.1 Зависимость от внутреннего API Apache Spark

Декорирование метода `MicroBatchExecution.runBatch` через инструментацию байт-кода (§3.2.2) опирается на внутреннее, не специфицированное публично API Apache Spark. Изменение сигнатуры метода, его разбиение на несколько методов или существенное изменение последовательности вызовов внутри него в новых минорных версиях Spark способно сломать декоратор: либо в момент загрузки агента (несовпадение сигнатуры), либо в момент исполнения (несовпадение предположений о порядке вызовов `Source.getBatch` и `Sink.addBatch`).

Принятые меры смягчения. Декорируемые классы Apache Spark вынесены в модуль `spark-adapter`, имеющий по реализации на каждую поддерживаемую версию Spark, и подгружаются Java-агентом по результатам определения версии в момент инициализации (§3.2.2). Конкретный декоратор `runBatch` для решения 2 размещается в существующей инфраструктуре адаптеров и опирается на стабильные точки взаимодействия с `Source` и `Sink`, наблюдаемые во всех версиях Spark, поддерживаемых SPYT (3.2.x–3.5.x). Расширение поддерживаемого диапазона версий Spark в SPYT выполняется централизованно через `spark-adapter` и не требует отдельной работы по сопровождению декоратора решения 2.

3.7.2 Общая грс-прокси как точка отказа

Конфигурация с общим пулом грс-прокси (`spark.ytsaurus.grc.job.proxy.enabled=false`) переносит точку отказа из `job-proxy` исполнителей в общий пул. Все запросы транзакции — `createTransaction`, `pingTransaction`, `modifyRows` с каждого исполнителя, `advanceConsumer`, `commit` — обслуживаются одним грс-прокси, на котором открыта транзакция, и отказ этого прокси в течение жизни транзакции приводит к её аварийному завершению независимо от состояния запросов записи на исполнителях. Тем не менее аварийное завершение транзакции не нарушает корректности: незафиксированная транзакция не оставляет следа ни в выходной таблице, ни в состоянии потребителя, и Spark переисполняет батч штатным образом. Кроме того, общий пул грс-прокси YTsaurus состоит из множества равноправных экземпляров с собственным резервированием на уровне инфраструктуры YTsaurus, и среднее время между отказами одного прокси из пула существенно превосходит соответствующую величину для одиночного процесса исполнителя. На уровне SPYT дополнительной логики восстановления соединения не вводится; повторное открытие транзакции выполняется на следующем батче после переисполнения текущего, что согласуется с принятой моделью отказоустойчивости Spark Structured Streaming.

3.7.3 Соотношение времени жизни таблет-транзакции и длительности батча

Таблет-транзакция YTsaurus имеет ограниченное время жизни, продлеваемое периодическими сообщениями `pingTransaction` (§2.3.3). Если обработка батча превышает таймаут транзакции и пинги не отправляются вовремя, прокси отменяет транзакцию автоматически, что наблюдается на драйвере как ошибка «No such transaction» при попытке `commit`.

Источниками превышения являются: длительные пользовательские преобразования внутри `runBatch`, медленная сериализация выходных строк, замедленная сеть до прокси, длинные паузы сборки мусора на драйвере. Дополнительный риск — несоответствие между временем жизни транзакции, заданным при её открытии, и реальной длительностью батча на загруженных кластерах.

Принятые меры смягчения. Во-первых, пинг транзакции запускается в отдельном фоновом потоке драйвера сразу после открытия транзакции и останавливается при `commit`, `abort` или исключении в декораторе. Период пинга задаётся в долях от заявленного времени жизни транзакции. Во-вторых, время жизни транзакции конфигурируется параметром, доступным пользователю SPYT, и устанавливается с запасом относительно ожидаемой 99-й перценти длительности батча. В-третьих, при наблюдении регулярных таймаутов в эксплуатации увеличение времени жизни транзакции является штатной реакцией, не требующей изменения кода.

3.7.4 Разрыв соединения исполнителя с грс-прокси

Между моментами `attachTransaction` и завершением последнего `modifyRows` исполнитель удерживает соединение с грс-прокси, на котором открыта транзакция. Разрыв этого соединения наблюдается на исполнителе как сетевая ошибка при очередном `modifyRows` либо как ошибка ответа от прокси.

Принятые меры смягчения. Во-первых, исполнитель не пытается восстановить соединение или повторно присоединиться к транзакции: исключение, выброшенное из замыкания `foreachPartition`, передаётся в драйвер штатным механизмом Spark и обрабатывается как сбой задачи записи. Во-вторых, драйвер при первом сбое любой задачи записи отменяет транзакцию через `abort` и не пытается фиксировать частично завершённую запись. В-третьих, переисполнение батча выполняется штатным механизмом Spark Structured Streaming с тем же набором смещений источника, поскольку `Source.commit` не успел выполнить продвижение смещения. Корректность сохраняется в силу того, что отменённая таблица-транзакция не оставляет видимых строк в выходной таблице (§2.2.1).

3.7.5 Сводка компромиссов

Совокупность принятых при проектировании решений приводит к набору компромиссов, каждый из которых обменивает одно желательное свойство системы на другое и разрешается за счёт инфраструктурных или эксплуатационных мер.

- В обмен на сохранение параллелизма записи допускается зависимость от внутреннего API Apache Spark, разрешаемая инфраструктурой версионных адаптеров `spark-adapter`.
- В обмен на единственность узкого места допускается централизация запросов транзакции на общем грс-прокси, разрешаемая опорой на серверное резервирование YTsaurus.
- В обмен на простоту обработки сбоев допускается отсутствие частичного восстановления внутри батча: любой сбой приводит к отмене транзакции и переисполнению батча целиком, что согласуется с моделью Spark Structured Streaming.

4 РЕАЛИЗАЦИЯ

В настоящей главе описывается реализация решения 2, выбранного в §3.4.4. Изложение следует снизу вверх: сначала рассматриваются вспомогательные компоненты (контекст транзакции, расширение `writer-a`), затем модифицированные потоковые компоненты (источник, приёмник), и наконец — управляющий слой (декоратор `MicroBatchExecution`, SPI-интерфейс и его реализация).

4.1 Структура изменений в репозитории SPYT

Реализация затрагивает пять модулей Gradle-проекта SPYT.

data-source — базовый модуль записи в динамические таблицы. Изменение: класс `YtDynamicTableWriter` (листинг A.7) расширяется необязательным параметром родительской транзакции.

data-source-extended — модуль потоковой обработки. Изменения: модификация `YtStreamingSource.commit` (листинг A.5) и `YtStreamingSink.addBatch` (листинг A.6); новые классы `YtStreamingTransactionContext` (листинг A.3) и `YtsaurusStreamingTransactionSupport` (листинг A.2).

spark-adapter — модуль версионных адаптеров Spark. Изменение: новый трейт `StreamingTransactionSupport` и вспомогательный трейт `StreamingTransactionHandle` в модуле `api` (листинг A.1).

spark-patch — модуль инструментации байт-кода. Изменение: новый декоратор `MicroBatchExecutionDecorators` (листинг A.4), размещённый в `spark-adapter/impl/spark-3.5.0` и применяемый ко всем поддерживаемым версиям Spark через механизм `@Decorate`.

yt-wrapper — модуль утилит для работы с YTsaurus. Изменение: метод `attachTransaction` в трейте `YtTransactionUtils` (листинг A.8), оборачивающий операцию `AttachTransaction` клиентской библиотеки YT.

4.2 Контекст транзакции

Передача идентификатора транзакции и адреса грс-прокси между методами драйвера реализована через `YtStreamingTransactionContext` (листинг A.3) — объект-синглтон, инкапсулирующий `InheritableThreadLocal`.

Хранимый тип — `case class StreamingTxContext(txId: String, stickyAddress: Option[String])`. Использование `InheritableThreadLocal` вместо `ThreadLocal` обеспечивает видимость контекста в дочерних потоках, порождаемых Spark при планировании задач.

Помимо контекста транзакции, объект хранит атомарный флаг `recoveryNeededFlag: AtomicBoolean`. Флаг устанавливается декоратором при сбое батча и сигнализирует о том, что состояние потребителя YTsaurus может расходиться с `commit`-логом Spark. При следующем батче декоратор логирует предупреждение и сбрасывает флаг после успешной фиксации.

Публичный API объекта:

```
object YtStreamingTransactionContext {
  def get: Option[StreamingTxContext]
  def setContext(ctx: StreamingTxContext): Unit
  def currentTransactionId: Option[String]
  def currentStickyProxyAddress: Option[String]
  def clearTransactionId(): Unit
  def isRecoveryNeeded: Boolean
  def markRecoveryNeeded(): Unit
  def clearRecoveryNeeded(): Unit
}
```

Жизненный цикл контекста в рамках одного батча: декоратор устанавливает контекст перед вызовом оригинального `runBatch` и очищает его в блоке `finally` после завершения батча — независимо от того, завершился ли батч успешно или с ошибкой.

4.3 Модификация потокового источника

4.3.1 Метод `getBatch`

Метод `YtStreamingSource.getBatch` (листинг A.5) не изменяется. Он формирует `YtQueueRDD` по диапазонам смещений и возвращает `DataFrame`. Смещение конца батча (`end: Offset`) передаётся Spark-ом в метод `commit` стандартным механизмом и не требует дополнительного сохранения в `ThreadLocal`.

4.3.2 Метод `commit`

Метод `YtStreamingSource.commit` реализует два режима работы в зависимости от наличия активного контекста транзакции.

Транзакционный режим (контекст установлен декоратором): смещение потребителя продвигается в составе уже открытой транзакции. Метод читает `txId` и `stickyAddress` из `YtStreamingTransactionContext`, при наличии `stickyAddress` создаёт клиент, зафиксированный на конкретном грс-прокси, и вызывает `offsetProvider.advance(..., parentTransactionId)`. Операция `advanceConsumer` выполняется в той же таблет-транзакции, что и запись данных на исполнителях.

Нетранзакционный режим с включённым флагом (контекст отсутствует, но `spark.yt.streaming.transactional=true`): метод завершается без действий. Это состояние возникает при сбое батча до установки контекста — транзакция не была открыта, и продвигать смещение не нужно.

Нетранзакционный режим (флаг выключен): штатное поведение — `offsetProvider.advance(..., None)` без транзакции.

Полный исходный код класса `YtStreamingSource` приведён в листинге A.5.

4.4 Модификация потокового приёмника

Метод `YtStreamingSink.addBatch` (листинг A.6) расширен поддержкой транзакционного режима. Изменения сосредоточены в двух местах: подготовка широковебательных переменных и создание клиента `YTsaurus` на исполнителе.

На драйвере метод читает `YtStreamingTransactionContext.get` и извлекает `txId` и `stickyAddress`. Оба значения упаковываются в широковебательные переменные `bcParentTransactionId` и `bcStickyProxyAddress`:

```
val txContext = YtStreamingTransactionContext.get
val bcParentTransactionId = txContext.map(ctx =>
    sparkContext.broadcast(ctx.txId))
val bcStickyProxyAddress =
    txContext.flatMap(_.stickyAddress).map(sparkContext.broadcast)
```

На исполнителе в замыкании `foreachPartition` выбор клиента `YTsaurus` зависит от наличия `stickyAddress`. При его наличии создаётся клиент, зафиксированный на конкретном грс-прокси (`YtClientProvider.ytClientFixedProxy`), что гарантирует направление запросов `attachTransaction` и `modifyRows` на тот же прокси, где открыта транзакция. При отсутствии — используется клиент из общего пула:

```
implicit val partitionYtClient: CompoundClient = stickyAddr match {
  case Some(addr) =>
    YtClientProvider.ytClientFixedProxy(bcYtClientConfiguration.value,
    addr)
  case None =>
    YtClientProvider.ytClient(bcYtClientConfiguration.value)
```

```

}
val attachedTx = txId.map(id => YtWrapper.attachTransaction(id))
val dynamicTableWriter = new YtDynamicTableWriter(bcPath.value,
    bcSchema.value,
    bcWriterConfig.value, bcParameters.value, attachedTx)

```

Объект `attachedTx: Option[ApiServiceTransaction]` передаётся в конструктор `YtDynamicTableWriter`. При его наличии все запросы `modifyRows` направляются в присоединённую транзакцию; при отсутствии — поведение не изменяется относительно нетранзакционного режима.

4.5 Расширение `YtDynamicTableWriter`

Класс `YtDynamicTableWriter` (листинг A.7) расширен необязательным параметром `parentTransaction: Option[ApiServiceTransaction] = None`:

```

class YtDynamicTableWriter(richPath: YPathEnriched, schema:
    StructType,
    wConfig: SparkYtWriteConfiguration, options: Map[String, String],
    parentTransaction: Option[ApiServiceTransaction] = None)
    (implicit ytClient: CompoundClient) extends OutputWriter

```

Параметр имеет значение по умолчанию `None`, что обеспечивает полную обратную совместимость: все существующие вызовы конструктора без явного указания транзакции продолжают работать без изменений.

Внутри класса параметр используется в методе `commitBatch`:

```

private def commitBatch(): Unit = {
    val request = modifyRowsRequestBuilder.build()
    YtWrapper.insertRows(request, parentTransaction)
    initBatch()
}

```

Метод `YtWrapper.insertRows` (листинг A.9) принимает `Option[ApiServiceTransaction]`: при `Some(tx)` запрос `ModifyRows` направляется через объект транзакции (`tx.modifyRows(request)`), при `None` — через обычный клиент `YTsaurus`.

Батчирование записи сохраняется: строки накапливаются в `ModifyRowsRequest.Builder` до достижения `wConfig.dynBatchSize`, после чего выполняется `commitBatch`. Параметр `dynBatchSize` управляется конфигурацией `spark.yt.write.dynBatchSize` и позволяет регулировать объём одного запроса `modifyRows` в рамках транзакции.

С точки зрения масштабируемости, батчирование играет ключевую роль. Поскольку все исполнители направляют запросы `modifyRows` в одну общую грс-прокси, количество запросов к прокси прямо влияет на нагрузку на неё. Слишком малое значение `dynBatchSize` приводит к большому числу мелких запросов, увеличивая накладные расходы на сетевое взаимодействие и обработку на прокси. Слишком большое значение увеличивает размер каждого запроса, что может привести к росту задержек и потреблению памяти на исполнителях. Оптимальное значение зависит от характеристик сети, производительности грс-прокси и объёма данных в батче, и подбирается экспериментально для конкретной конфигурации кластера.

4.6 Декоратор MicroBatchExecution

Декоратор `MicroBatchExecutionDecorators` (листинг А.4) подменяет метод `runBatch` класса `MicroBatchExecution` через механизм инструментации байт-кода `spark-patch`. Аннотации `@Decorate` и `@OriginClass` указывают агенту целевой класс и метод для подмены; оригинальный метод остаётся доступным под именем `__runBatch`.

При выключенном флаге `spark.yt.streaming.transactional` декоратор прозрачно делегирует вызов оригинальному методу `__runBatch` и не вносит накладных расходов.

При включённом флаге последовательность действий следующая.

1. Через SPI создаётся таблет-транзакция; её идентификатор и адрес грс-прокси сохраняются в `YtStreamingTransactionContext`.
2. Вызывается оригинальный `__runBatch`, внутри которого Spark вызывает `Source.getBatch`, планирует и исполняет задания записи через `Sink.addBatch`.
3. После успешного возврата из `__runBatch` декоратор вызывает `commitOffsets(availableOffsets)`, который обходит все источники и вызывает `source.commit(offset)`. В `YtStreamingSource.commit` смещение потребителя продвигается в составе открытой транзакции.
4. Транзакция фиксируется вызовом `currentTransaction.commit()`.
5. Флаг `recoveryNeeded` сбрасывается.

При исключении на любом из шагов 2–4:

- устанавливается флаг `recoveryNeeded`;
- транзакция отменяется (`abort`);
- если исключение произошло после записи в `commit`-лог Spark (`commitLogWritten = true`), запись откатывается через `commitLog.purgeAfter(batchId - 1)`, чтобы Spark переисполнил батч с теми же смещениями.

Вспомогательный метод `commitOffsets` вынесен в объект-компаньон и вызывает `source.commit` только для источников типа `Source (V1 API)`, что соответствует реализации `YtStreamingSource`.

4.7 SPI-точка расширения

4.7.1 Интерфейс StreamingTransactionSupport

Интерфейс `StreamingTransactionSupport` (листинг А.1) объявляет контракт жизненного цикла распределённой транзакции:

```
trait StreamingTransactionSupport {  
  def isTransactionalStreamingEnabled(sparkSession: SparkSession):  
    Boolean  
  def createTransaction(sparkSession: SparkSession):  
    StreamingTransactionHandle  
  def setTransaction(handle: StreamingTransactionHandle): Unit  
  def setTransactionId(txId: String): Unit  
  def setStickyProxyAddress(address: Option[String]): Unit  
  def clearTransactionId(): Unit  
  def isRecoveryNeeded: Boolean  
  def markRecoveryNeeded(): Unit  
  def clearRecoveryNeeded(): Unit  
}
```

Вспомогательный трейт `StreamingTransactionHandle` (там же, листинг А.1) представляет открытую транзакцию:

```
trait StreamingTransactionHandle {
  def getId: String
  def getStickyProxyAddress: Option[String]
  def commit(): Unit
  def abort(): Unit
}
```

Загрузка реализации выполняется через `ServiceLoader` при первом обращении:

```
object StreamingTransactionSupport {
  lazy val instance: StreamingTransactionSupport =
    ServiceLoader.load(classOf[StreamingTransactionSupport]).findFirst().get()
}
```

Размещение интерфейса в модуле `spark-adapter/api` (а не в `data-source-extended`) обусловлено тем, что декоратор `MicroBatchExecutionDecorators` в `spark-patch` должен обращаться к нему без циклической зависимости.

4.7.2 Реализация `YtSaurusStreamingTransactionSupport`

Класс `YtSaurusStreamingTransactionSupport` (листинг A.2) регистрируется в META-INF/services/tech.ytsaurus.spyt.adapter.StreamingTransactionSupport.

Метод `createTransaction` создаёт таблет-транзакцию с параметром `sticky=true`, что соответствует `TransactionType.Tablet` в клиентской библиотеке YT. Полный исходный код класса приведён в листинге A.2.

Класс `YtStreamingTransactionHandle` (там же, листинг A.2) оборачивает `ApiServiceTransaction`. Адрес grpc-прокси, на которой открыта транзакция, получается через рефлексия — метод `getRpcProxyAddress` не входит в публичный API `ApiServiceTransaction`, поэтому вызывается через `getDeclaredMethods` с `setAccessible(true)`. При недоступности метода (например, в будущих версиях клиентской библиотеки) возвращается `None`, и исполнители используют клиент из общего пула без фиксации на конкретном прокси.

Методы `commit` и `abort` делегируют вызовы объекту `ApiServiceTransaction` с блокирующим ожиданием (`join()`). В `abort` перехватывается `IllegalStateException`, возникающий при попытке отменить уже завершённую транзакцию.

4.8 Конфигурация и обратная совместимость

Транзакционный режим управляется конфигурационным параметром `spark.yt.streaming.transactional` (тип `Boolean`, значение по умолчанию `false`). Параметр объявлен в объекте `SparkYtConfiguration.Streaming` (листинг A.11):

```
object Streaming {
  case object Transactional extends ConfigEntry[Boolean]
    ("streaming.transactional", Some(false))
}
```

При значении `false` (по умолчанию) поведение всех компонентов идентично поведению до введения транзакционного режима:

- декоратор `MicroBatchExecutionDecorators` прозрачно делегирует вызов оригинальному `runBatch` без создания транзакции;
- `YtStreamingSource.commit` выполняет штатное продвижение смещения без транзакции;

- `YtStreamingSink.addBatch` не читает контекст транзакции;
`bcParentTransactionId` и `bcStickyProxyAddress` равны `None`;
`YtDynamicTableWriter` создаётся с `parentTransaction = None`.

Таким образом, существующие потоковые задания, не устанавливающие параметр `spark.yt.streaming.transactional=true`, продолжают работать без изменений и без каких-либо накладных расходов, связанных с транзакционным режимом.

Для включения транзакционного режима дополнительно требуется конфигурация `spark.ytsaurus.rpc.job.proxy.enabled=false`, переключающая исполнители на общий пул rpc-прокси кластера. Без этой настройки исполнители используют локальные `job-proxy`, изолированные от прокси драйвера, и операция `attachTransaction` завершится ошибкой «No such transaction» (см. §2.4.2).

5 ТЕСТИРОВАНИЕ

В настоящей главе описывается проверка корректности и производительности реализации, выбранной в §3.4.4. Изложение начинается с обзора существующих автоматизированных тестов модуля `data-source-extended`, затем переходит к экспериментальной части — серии бенчмарков, разработанных для эмпирического сравнения транзакционного и нетранзакционного режимов.

5.1 Стратегия тестирования

Применяется трёхуровневая стратегия:

- **Модульные тесты** проверяют отдельные компоненты в изоляции с использованием `mock`-объектов: алгоритмы парсинга и сериализации смещений, логика `latestOffset/getBatch` источника, поведение `YtStreamingSink.addBatch` и `YtStreamingSource.commit` под управлением `YtStreamingTransactionContext`. Цель — зафиксировать инварианты на уровне отдельных классов и сделать сбой при их нарушении локализованным.
- **Интеграционные тесты** запускают реальный `SparkSession` и обращаются к локально поднимаемому `Ytsaurus`-кластеру. Проверяются сквозные сценарии: чтение из очереди, запись в результирующую таблицу, продвижение потребителя, восстановление после сбоев, многоисточниковая конфигурация.
- **Эксперименты на удалённом кластере** проводятся в выделенном пуле тестового `YT`-кластера. Их роль — измерение производительности на реалистичном объёме данных и оценка корректности строго однократной семантики в условиях, приближенных к продакшен-нагрузке.

5.2 Тестовая инфраструктура

Автоматизированные тесты расположены в пакете `tech.ytsaurus.spyt.streaming` модуля `data-source-extended`. Совокупный объём — 1863 строки `Scala`-кода в четырёх файлах:

- `YtQueueUtilsTest.scala` — 95 строк, 10 модульных тестов вспомогательных утилит работы со смещениями очереди.
- `YtStreamingSourceTest.scala` — 459 строк, 21 модульный тест источника `YtStreamingSource` с `mock`-провайдером смещений.
- `YtTransactionalStreamingTest.scala` — 210 строк, 4 теста транзакционного коммита и отката с локальным `YT`-клиентом.
- `YtsaurusStreamingTest.scala` — 1099 строк, 22 интеграционных теста полного цикла стриминга.

Базовые трейты LocalSpark и LocalYtClient обеспечивают единое окружение: SparkSession создаётся однократно на тестовый класс, локальный YT поднимается до прогона тестового набора, изолированные пути для каждого теста генерируются через `UUID.randomUUID()`. Поднятие реального YT-кластера в Docker позволяет проверять не только логику Spark-кода, но и взаимодействие с таблет-сервисом, продвижение потребителя и схему динамических таблиц.

Эксперименты на удалённом кластере выполняются в окружении SPYT 2.9.1, в выделенном пуле тестового YT-кластера. Конфигурация исполнителей: 2 executors × (2 cores / 6 GB heap + 2 GB overhead), драйвер 2 cores / 3 GB.

Характеристика тестовых данных. В качестве источника используется живая упорядоченная очередь, в которую активный продьюсер непрерывно дописывает события журнала аналитической СУБД (журнал исполненных SQL-запросов). Значимые для тестирования параметры источника:

- *Активный продьюсер:* запись в очередь происходит одновременно с чтением, что моделирует реальную потоковую нагрузку (а не batch-замер на статическом срезе).
- *Многотаблетная организация:* очередь распределена по нескольким таблетам, что позволяет проверить корректность параллельного чтения и независимого продвижения смещений по партициям.
- *Сложная схема:* около 85 колонок, представляющих смесь типов: скалярные идентификаторы и временные метки, числовые метрики, текстовые поля, а также сложные типы — Map (счётчики и настройки) и Array (списки имён таблиц, потоков, словарей).
- *Кратность событий на ключ:* на один логический запрос в очереди порождается несколько строк (события начала и завершения), что позволяет проверить корректность работы с повторяющимися значениями натурального ключа.
- *Часть колонок исключается на стороне SPYT:* колонки `transaction_id` и `http_headers` имеют типы, не сериализуемые штатным путём, и отбрасываются на чтении.

Иллюстративный пример формы данных (значения приближены и не соответствуют конкретным записям источника):

```
query_id           : "8a3f4d7c-12b9-4e6a-91d3-..."
event_time         : 2024-06-15T14:32:08.421Z
event_time_microseconds : 1718462528421739
type               : "QueryFinish"
query_duration_ms  : 142
read_rows          : 18 432
read_bytes         : 2 097 152
memory_usage       : 8 388 608
normalized_query_hash : 14792038418273645210
query              : "SELECT ... FROM table_a WHERE ..."
ProfileEvents      : {"SelectedRows": 18432,
"ReadBufferFromFileDescriptor": 12, ...}
Settings           : {"max_threads": "8", "readonly": "0", ...}
tables             : ["db.table_a", "db.table_b"]
databases          : ["db"]
```

В транзакционном режиме результирующая таблица — упорядоченная динамическая, схема повторяет схему очереди (за исключением отброшенных колонок). В нетранзакционном режиме — сортированная динамическая, к схеме добавляются два ключевых столбца `__sptyt_streaming_src_tablet_index` и `__sptyt_streaming_src_row_index`, обеспечивающие естественную идемпотентность записи при возможных повторах.

Целевой объём прогона — 1 000 000 строк по сумме смещений потребителя по всем партициям.

5.3 Модульные тесты

5.3.1 Утилиты работы со смещениями

YtQueueUtilsTest проверяет инварианты класса `YtQueueOffset` и связанной логики продвижения потребителя:

- *Сериализация и десериализация смещений*: проверяется однострочное JSON-представление, упорядочение партиций при сериализации (стабильность снимка), корректность повторной сериализации/десериализации.
- *Валидация диапазонов*: тест «throw exception when partition list is sparse» гарантирует отказ при пропусках в списке партиций — инвариант, на который опирается алгоритм построения `YtQueueRange`.
- *Идемпотентность advanceConsumer*: тесты «not advance consumer on commit if new offset is less than last committed» и «...is more than max offset» фиксируют, что повторный коммит не двигает смещение назад и не позволяет «перепрыгнуть» вперёд через неподтверждённые строки. Это основа корректного восстановления.
- *Обработка ошибок транзакции*: тест «throw an error when the transaction has expired» проверяет, что истечение таблет-транзакции при коммите порождает исключение, а не молчаливо теряет данные.

5.3.2 Логика источника

YtStreamingSourceTest содержит 21 тест, покрывающий ветви `latestOffset`, `getBatch` и `commit` источника `YtStreamingSource`. Используется mock-провайдер смещений (`mockito`), позволяющий моделировать произвольное состояние потребителя и реакцию YT-сервиса.

Группы тестов:

- *Транзакционный режим* (5 тестов): проверяется приоритет смещения потребителя над контрольной точкой Spark при расхождении, использование `YtStreamingTransactionContext` при `commit`, отсутствие вызова `advance` при отсутствии транзакционного контекста.
- *Нетранзакционный режим* (4 теста): проверяется поведение при рассинхронизации контрольной точки и потребителя, возврат к состоянию потребителя при отставании Spark, переисполнение батча с тем же диапазоном. Один из тестов (с маркировкой BUG REPR0) фиксирует ранее обнаруженный дефект: продвижение от Spark `startOffset` на втором батче при пустом потребителе — он используется как защита от регрессии после фикса.
- *ReadLimit и admission control* (3 теста): проверяется построение `CompositeReadLimit` при заданной опции `max_rows_per_partition` и независимое ограничение каждой партиции в многотаблетной очереди.
- *Кеширование maxOffset* (3 теста): проверяется отсутствие отката кешированного значения при временной недоступности YT-сервиса и корректная инвалидация кеша.
- *Построение YtQueueRange* (3 теста): проверяется создание одного диапазона на партицию, обработка пустого батча (`start == end`), десериализация `SerializedOffset` при чтении из `commit`-лога.
- *Контрактные ограничения* (3 теста): проверяется, что вызов устаревшего `getOffset` (v1 API) порождает исключение — поддерживается только `latestOffset` Spark v2.

5.3.3 Транзакционный приёмник и фиксация

YtTransactionalStreamingTest проверяет четыре ключевых сценария поверх локального YT-клиента, без поднятия полноценного Spark Streaming:

- *Передача идентификатора транзакции*: тест «pass parent transaction id to provider's advance on commit» проверяет, что `YtStreamingSource.commit`

извлекает txId из YtStreamingTransactionContext и передаёт его в advanceConsumer — это инвариант, обеспечивающий атомарность «вставка + продвижение потребителя» в одной таблет-транзакции.

- *Распространение исключений*: тест «propagate exception from provider's advance on transactional commit» проверяет, что сбой advance не глушится и приводит к отмене всей таблет-транзакции выше по стеку.
- *Успешный коммит*: тест «commit data successfully with transactional streaming» выполняет полный цикл: sink.addBatch(0, df) → source.commit(offset) → fixture.commitAndVerify(), после чего проверяет, что в результирующей таблице ровно 10 ожидаемых строк.
- *Откат при сбое*: тест «rollback data when commit fails» моделирует ошибку advanceConsumer и проверяет, что после fixture.abortSafely() в результирующей таблице **ноль строк** — данные, записанные addBatch, становятся невидимыми при отмене транзакции. Это эмпирическое подтверждение центральной гарантии транзакционного решения: незакоммиченный батч атомарно исчезает.

5.4 Интеграционные тесты

YtSaurusStreamingTest содержит 22 теста, выполняющих сквозные сценарии стриминга с реальным SparkSession и локально развёрнутым YT. Тесты сгруппированы по нескольким направлениям.

5.4.1 Базовые сценарии

- *Чтение и запись очереди* (тесты «write YT queue», «read YT queue», «run pipeline on YT queues»): проверяется минимальный сквозной путь — создание queue/consumer/result, запись тестовых данных в очередь, исполнение pipeline, проверка содержимого результирующей таблицы.
- *Ограничение размера батча* (тест «correct streaming with max_rows_per_partition»): проверяется работа admission control — корректная нарезка батчей при ограничении и корректное число итераций для дочитывания заданного объёма.
- *Корректность типов* (тесты «correctly stream unsigned byte, short and int types» и «correct streaming with custom YT types»): проверяется сохранение всех YT-типов при сериализации через стриминговый pipeline.

5.4.2 Гарантии повторной обработки

Группа тестов «one streaming launch» / «several streaming launches» проверяет поведение при повторном запуске запроса с той же контрольной точкой и тем же потребителем. Сравниваются гарантии «не реже одного раза» (нетранзакционный режим без служебных колонок) и строго однократной обработки (нетранзакционный со служебными колонками для дедупликации, и транзакционный режим). Отдельно проверяется сценарий «continue streaming after removing checkpoints with transaction»: при потере контрольной точки транзакционный режим должен восстановиться из состояния потребителя без дубликатов и без потерь.

5.4.3 Многоисточниковая конфигурация

- *Несколько очередей с одним результатом* (тесты «several sources-queues: check consumer and result synchronization» и «check full data delivery»): три потока, читающие из независимых очередей и пишущие в общую результирующую таблицу, проверяется полная доставка данных и согласованность смещений каждого потребителя.
- *Несколько приёмников из одной очереди* (тест «streaming with several sinks and consumers»): проверяется ветвление записи — одна очередь, три потребителя с отдельными результирующими таблицами, доставка идентичных данных во все три.

- *Многоисточниковая конфигурация с транзакциями* (тест «several sources-queues with transaction»): то же, что и базовый многоисточниковый тест, но с включённым транзакционным режимом — проверяется, что несколько параллельных таблет-транзакций сосуществуют без взаимных блокировок.

5.4.4 Поведение при сбоях

- *Размонтирование таблицы в процессе записи* (тест «exactly-once with transaction when result table is unmounted while queue is being written»): результирующая таблица размонтируется во время записи, ожидается падение StreamingQuery. После перемонтирования таблицы и перезапуска запроса проверяется, что в результирующей таблице ровно ожидаемое число строк (30) и что состояние потребителя согласовано с этим числом. Тест эмпирически подтверждает атомарность таблет-транзакции: записи незавершённого батча не видны после сбоя, и переисполнение даёт корректное число строк без дубликатов.
- *Десинхронизация смещений* (тест «streaming after offsets desynchronization: checkpoints are behind on consumer»): проверяется восстановление, когда Spark checkpoint отстаёт от потребителя — реализация приоритизирует consumer как авторитетный источник истины.

5.4.5 Тестовое покрытие требований

Связь между функциональными требованиями §3.1.1 и автоматизированными тестами:

Требование	Тесты
Чтение из очереди с продвижением потребителя	YTsaurusStreamingTest §5.4.1
Запись в динамическую таблицу	YTsaurusStreamingTest §5.4.1
Корректность batch boundaries (max_rows_per_partition)	YtStreamingSourceTest §5.3.2, YTsaurusStreamingTest §5.4.1
Идемпотентность advanceConsumer	YtQueueUtilsTest §5.3.1
Атомарность транзакционной фиксации	YtTransactionalStreamingTest §5.3.3, YTsaurusStreamingTest §5.4.4
Восстановление после сбоя в записи	YTsaurusStreamingTest §5.4.4
Несколько одновременных стримов	YTsaurusStreamingTest §5.4.3

5.5 Бенчмарки на удалённом кластере

Помимо автоматизированных тестов, для эмпирической оценки производительности и корректности на реалистичной нагрузке разработана серия Python-бенчмарков. Они расположены в [examples/python/benchmark_streaming/](#) и являются независимыми от тестовой инфраструктуры.

5.5.1 Бенчмарк производительности

Скрипт `main.py` выполняет один прогон стриминга от старта до достижения целевого объёма (1 000 000 строк, накопленных по сумме смещений потребителя). На каждом батче из `query.recentProgress` извлекаются метрики: `triggerExecutionMs`, `addBatchMs`, `numInputRows`, `inputRowsPerSecond`, `processedRowsPerSecond`. По завершении печатается сводная таблица и суммарные показатели.

Бенчмарк параметризован флагом `--tx-mode-enabled`, который активирует конфигурацию `spark.yt.streaming.transactional=true` и переключает схему результирующей таблицы между упорядоченной (для транзакционного режима) и

сортированной со служебными колонками `__splt_streaming_src_tablet_index`, `__splt_streaming_src_row_index` (для нетранзакционного режима с дедупликацией).

Прогоны проводились на одной и той же источниковой очереди (см. характеристику тестовых данных в §5.2). Транзакционная конфигурация требует отключения локального RPC job proxy (`spark.ytsaurus.rpc.job.proxy.enabled=false`) — это особенность текущей реализации SPYT 2.9.1, обусловленная требованиями к семантике `AttachTransaction`, и является частью самого режима, а не варьируемым параметром эксперимента.

Выполнено по четыре прогона каждого режима. Результаты приведены в таблицах 5.1 и 5.2.

Таблица 5.1 — Транзакционный режим, запись в упорядоченную динамическую таблицу

№	Wall time, c	Суммарное triggerExecution, мс	Суммарное addBatch, мс
1	728,50	710 231	594 564
2	606,05	593 496	441 935
3	940,85	902 319	770 139
4	727,76	697 327	549 483
Среднее	750,79	725 843	589 030

Таблица 5.2 — Нетранзакционный режим, запись в сортированную динамическую таблицу со служебными колонками

№	Wall time, c	Суммарное triggerExecution, мс	Суммарное addBatch, мс
1	786,06	766 979	724 617
2	757,73	729 661	687 939
3	667,79	659 697	617 765
4	819,09	784 941	737 001
Среднее	757,67	735 320	691 830

Каждый прогон обрабатывал ровно 20 батчей по 50 000 строк. Среднее значение `addBatchMs` в стационарном режиме (без учёта первого батча, отражающего стоимость инициализации) составило 27 200 мс для транзакционного режима против 31 900 мс для нетранзакционного.

Анализ результатов:

- Wall time двух режимов практически идентичен:** среднее 750,8 с против 757,7 с, разница $\approx 0,9$ %. На метрике сквозного времени режимы неразличимы при текущей конфигурации ресурсов.
- Транзакционный режим стабильно быстрее по addBatchMs:** среднее 589 030 мс против 691 830 мс, разница ≈ 15 %. В стационарном режиме — тот же зазор: 27 200 мс против 31 900 мс на батч. Транзакционный режим выиграл во всех четырёх парных сравнениях, что согласуется с теоретической моделью: одна таблет-транзакция «вставка + advance потребителя» дешевле двух отдельных RPC-вызовов с последующей дедупликацией на сортированной таблице.
- Расхождение между addBatchMs и wall time** объясняется наличием idle-промежутков между батчами в среднем по серии. На метрике, измеряющей именно стоимость записи, преимущество транзакционного режима проявляется

отчётливо; на метрике, включающей паузы и накладные расходы планировщика, оно маскируется внутри-режимной дисперсией.

4. **Пропускная способность** в обоих режимах находится в диапазоне 1 300–1 700 строк/с. Это значение определяется конфигурацией исполнителей и пропускной способностью YT-сервиса, а не выбором режима.
5. **Стоимость прогрева** значительна и одинакова в обоих режимах: первый батч занимает 48–110 с против 16–53 с в стационарном режиме. Для коротких задач прогрев доминирует, для длительных — амортизируется.
6. **Дисперсия per-batch $\approx 50\%$** . При этом средние величины по серии из четырёх прогонов уже разделяются по метрике `addBatchMs`. Стабильный знак разности во всех четырёх парах согласуется с теоретическим ожиданием и подтверждает гипотезу качественно. Для более строгой статистической оценки требуется увеличение числа прогонов до шести-восьми, что выходит за рамки настоящей работы.

5.5.2 Сравнительный sanity-анализ результирующих таблиц

Для подтверждения функциональной эквивалентности режимов в сценарии без сбоев разработан скрипт `compare_results_spark.py`. Он читает обе результирующие таблицы как Spark DataFrame, вычисляет:

- `COUNT(*)` каждой таблицы;
- `COUNT(DISTINCT __splt_streaming_src_*)` нетранзакционной таблицы для подсчёта дубликатов по `service-pk`;
- `COUNT(DISTINCT query_id)` каждой таблицы и размер пересечения;
- После обрезки нетранзакционной таблицы по множеству `query_id` транзакционной (полусоединением (`left semi join`)) — повторное сравнение в пределах общего окна чтения.

Результат для итоговых таблиц из §5.5.1:

```
tx COUNT(*) : 1 000 000
non-tx COUNT(*) (raw) : 1 040 000
non-tx COUNT(DISTINCT pk) (raw) : 1 040 000
non-tx duplicates by pk (raw) : 0
tx COUNT(DISTINCT query_id) : 500 031
non-tx COUNT(DISTINCT query_id) (raw) : 520 115
intersection (raw) : 500 031
only in tx (raw) : 0
only in non-tx (raw) : 20 084
non-tx COUNT(*) (trimmed) : 1 000 000
non-tx COUNT(DISTINCT query_id) (trimmed) : 500 031
intersection (trimmed) : 500 031
only in tx (trimmed) : 0
only in non-tx (trimmed) : 0
Verdict: MATCH on query_id after trimming
```

Интерпретация:

1. **Корректность нетранзакционного режима в сценарии без сбоев:** `COUNT(*) = COUNT(DISTINCT __splt_streaming_src_*)`, дубликатов нет. В отсутствие сбоев дедупликация по служебным колонкам не активируется, нетранзакционный режим записывает каждую исходную строку ровно один раз.
2. **Корректность транзакционного режима:** записано ровно 1 000 000 строк — точное совпадение с целевым объёмом эксперимента.
3. **Эквивалентность данных:** все 500 031 уникальных значения `query_id`, присутствующих в транзакционной таблице, присутствуют и в нетранзакционной (`only-in-tx = 0`). После обрезки нетранзакционной

таблицы до общего множества query_id получено ровно 1 000 000 строк с тем же множеством уникальных идентификаторов — поэлементное равенство содержимого.

4. **Расхождение по числу строк** между «сырыми» таблицами (1 000 000 против 1 040 000) обусловлено зернистостью останова бенчмарка: проверка достижения целевого смещения выполняется с периодом 30 секунд, между двумя проверками может пройти дополнительный батч объёмом 50 000 строк. Это артефакт измерительного скрипта, не связанный с семантикой режимов.

Sanity-анализ подтверждает функциональную эквивалентность транзакционного и нетранзакционного режимов на штатном пути исполнения.

5.5.3 Проверка строгой однократности при сбое

Для эмпирического подтверждения строго однократной семантики транзакционного режима и корректной работы дедупликации нетранзакционного режима в условиях сбоев разработан скрипт [failure_injection.py](#).

Методика. Скрипт запускается как обычный Spark-стриминг, но каждые 60 секунд вызывает `StreamingQuery.stop()` и немедленно стартует новый запрос с тем же путём к контрольной точке. Между остановками Spark исполняет несколько микропакетов; момент остановки в среднем приходится на середину одного из батчей, что моделирует сбой драйвера и вынуждает Spark переисполнить незавершённый батч после рестарта. Цикл продолжается до достижения целевого объёма 200 000 обработанных строк. По завершении подсчитываются: общее число строк в результирующей таблице (`COUNT(*)`), число уникальных значений служебного первичного ключа (`__spty_streaming_src_tablet_index`, `__spty_streaming_src_row_index`) для нетранзакционного режима, и число уникальных пар (`query_id`, `type`) — комбинации, гарантированно уникальной на каждое исходное событие очереди.

Критерии корректности. Для транзакционного режима критерием PASS является равенство `COUNT(*) = COUNT(DISTINCT (query_id, type))`: повторно записанные строки от переисполнённых батчей выявились бы как превышение `COUNT(*)` над числом уникальных событий. Для нетранзакционного режима — равенство `COUNT(*) = COUNT(DISTINCT (__spty_streaming_src_*))`: физическая дедупликация на сортированной таблице сохраняет идемпотентность даже при повторной записи. Превышение `COUNT(*)` над `final consumer offset` в обоих режимах являлось бы нарушением строго однократной обработки.

Результаты. Прогоны проведены на свежесозданных каталогах для каждого режима, чтобы исключить остаточное состояние. Результаты приведены в таблице 5.3.

Таблица 5.3 — Результаты прогона failure_injection в двух режимах

Показатель	Транзакционный режим	Нетранзакционный режим
Число принудительных остановок	5	5
Целевой объём (target_rows)	200 000	200 000
Финальное смещение потребителя	200 000	200 000
COUNT(*) результирующей таблицы	200 000	200 000
COUNT(DISTINCT (__spty_streaming_src_*))	n/a (нет колонок)	200 000
Дубликатов по служебному ключу	n/a	0
COUNT(DISTINCT (query_id, type))	200 000	200 000
Дубликатов по натуральному ключу	0	0

Показатель	Транзакционный режим	Нетранзакционный режим
Общее время прогона, с	285,71	268,43
Вердикт	PASS	PASS

Анализ результатов. В обоих режимах при пяти принудительных рестартах результирующая таблица содержит ровно столько строк, сколько было прочитано из очереди, и каждая строка имеет уникальное значение натурального ключа. Это означает, что ни одна строка не записалась в выходную таблицу дважды от переисполнённого батча и ни одна строка не была потеряна. Дополнительная проверка по служебному ключу нетранзакционного режима показала отсутствие дубликатов и на физическом уровне ($\text{COUNT}(\ast) = \text{COUNT}(\text{DISTINCT pk})$), то есть дедупликация по сортированной таблице успела отработать в каждом случае, когда переисполненный батч повторно отправил строки в выходную таблицу.

Полученные результаты эмпирически подтверждают корректность реализации в обоих режимах: транзакционный режим обеспечивает строго однократную обработку по построению — за счёт атомарной фиксации записи и продвижения смещения в одной таблет-транзакции; нетранзакционный режим — за счёт идемпотентной записи в сортированную таблицу с уникальным первичным ключом, формируемым из координат исходного сообщения в очереди. Время прогона в обоих режимах сопоставимо (285,7 с против 268,4 с), что согласуется с результатами раздела 5.5.1: на штатном пути исполнения разница режимов по производительности невелика и в полной мере проявляется только на метрике `addBatchMs` без учёта пауз между батчами.

5.6 Выводы по главе

Проверка корректности и производительности реализации проведена на трёх уровнях: модульное тестирование (35 тестов в трёх файлах с использованием `mock-объектов`), интеграционное тестирование (22 теста на локальном YТ-кластере), бенчмарки на удалённом кластере (4 + 4 прогона производительности, `sanity-сравнение` по натуральному ключу, две серии прогонов с принудительными сбоями).

Результаты экспериментов на удалённом кластере подтверждают теоретические ожидания. На метрике производительности транзакционный режим обеспечивает строгую однократность записи без накладных расходов на дедупликацию и в среднем на 15 % меньшее время записи батча. `Sanity-сравнение` по `queue_id` после обрезки до общего окна чтения показало поэлементное равенство данных, записанных двумя режимами, что подтверждает функциональную эквивалентность реализаций в отсутствие сбоев. Прогон с принудительными остановками драйвера в обоих режимах (пять рестартов за прогон) дал ровно 200 000 уникальных строк на 200 000 прочитанных событий, без единого дубликата и без потерь, что эмпирически подтверждает строго однократную обработку на стороне приёмника как при транзакционной фиксации, так и при идемпотентной записи в сортированную таблицу.

Автоматизированное тестовое покрытие позволяет ловить регрессии на этапе разработки: все ключевые инварианты (идемпотентность `advanceConsumer`, атомарность таблет-транзакции, приоритет потребителя как источника истины при рассинхронизации контрольной точки, корректное переисполнение батча после сбоя в записи) защищены тестами, которые исполняются в CI при каждом изменении.

ЗАКЛЮЧЕНИЕ

В рамках настоящей выпускной квалификационной работы решена задача обеспечения гарантии строго однократной потоковой обработки данных в Apache Spark при записи в динамические таблицы YTsaurus. Все поставленные во введении

задачи выполнены.

Проанализирована архитектура Apache Spark Structured Streaming и исходная реализация потокового адаптера SPYT с точки зрения семантики доставки. Показано, что разделение ответственности между источником и приёмником на штатных интерфейсах Spark приводит к рассогласованию между записью данных в выходную таблицу и продвижением смещения потребителя в случае сбоя, а существующее в SPYT частичное решение, основанное на идемпотентной записи в отсортированные динамические таблицы, ограничено по области применимости и по пропускной способности.

Изучена модель транзакций YTsaurus. Установлено, что таблет-транзакции YTsaurus являются единственным механизмом, в котором атомарно объединяются запись пользовательских данных в выходную динамическую таблицу и продвижение смещения потребителя очереди. Зафиксировано принципиальное ограничение реализации: состояние таблет-транзакции локально для конкретного RPC-прокси и не реплицируется между прокси кластера. Из этого ограничения выведены два допустимых пути организации распределённой транзакции: сведение всего батча к одному исполнителю либо переход на общий пул RPC-прокси с поддержкой клиентской библиотекой операции присоединения к существующей транзакции.

Спроектирована архитектура модуля интеграции на основе второго пути — общего пула RPC-прокси с операцией `attachTransaction`. Решение обосновано в §3.4.4 сравнительной таблицей с двумя альтернативами: оно сохраняет параллелизм записи, не зависит от объёма результирующей таблицы и обеспечивает строго однократную обработку при произвольных пользовательских преобразованиях.

Реализован механизм распределённой транзакции в исходном коде SPYT с использованием трёх точек расширения Apache Spark: интерфейса `StreamingTransactionSupport`, регистрируемого через стандартный механизм `ServiceLoader`; декоратора метода `MicroBatchExecution.runBatch`, внедряемого средствами инструментации байт-кода через Java-агент; контекста `YtStreamingTransactionContext` на основе `ThreadLocal`. Изменения распределены между модулями `data-source`, `data-source-extended`, `spark-adapter`, `spark-patch` и `yt-wrapper`. Переключение между исходной семантикой «не реже одного раза» и новой строго однократной семантикой выполняется одним конфигурационным параметром `spark.yt.streaming.transactional` без модификации пользовательского кода потокового запроса.

Разработан набор тестов, подтверждающих корректность семантики строго однократной обработки. На уровне модульного тестирования покрыты инварианты вспомогательных утилит работы со смещениями, ветви методов `latestOffset`, `getBatch` и `commit` источника, контракт транзакционного приёмника при успешном коммите и при откате (35 тестов). На уровне интеграционного тестирования покрыты сквозные сценарии чтения из очереди и записи в результирующую таблицу, многоисточниковая конфигурация, восстановление после сбоев в виде размонтирования таблицы и рассинхронизации смещений (22 теста). Все тесты исполняются в непрерывной интеграции при каждом изменении исходного кода.

Произведена эмпирическая оценка реализации на удалённом тестовом YT-кластере. Серия из четырёх парных прогонов транзакционного и нетранзакционного режимов на источнике объёмом 1 000 000 строк показала: транзакционный режим стабильно демонстрирует меньшее суммарное время записи батча (среднее 589 030 мс против 691 830 мс, преимущество $\approx 15\%$); транзакционный режим выиграл по этой метрике во всех четырёх парных сравнениях; пропускная способность обоих режимов сопоставима по сквозному времени прогона. Сравнение результирующих таблиц по натуральному ключу подтвердило поэлементную эквивалентность данных, записанных двумя режимами на общем окне чтения.

Практическая значимость работы состоит в том, что разработанный модуль может быть напрямую использован в производственных потоковых конвейерах, построенных на YTsaurus и SPYT, для перехода с гарантии «не реже одного раза» к строго однократной без изменения пользовательского кода и без введения

искусственных идемпотентных ключей в пользовательскую модель данных. Реализация интегрирована в кодовую базу SPYT и доступна пользователям проекта с версии 2.9.1.

К ограничениям полученного решения относятся: зависимость от внутреннего API Apache Spark, разрешаемая инфраструктурой версионных адаптеров; необходимость использования общего пула RPC-прокси, что переносит точку отказа на этот пул; ограниченное время жизни таблет-транзакции, которое должно превышать длительность обработки одного батча и конфигурируется отдельным параметром.

Направления дальнейшей работы. Во-первых, эмпирическая оценка реализации может быть расширена двумя дополнительными классами экспериментов: проверкой строго однократной обработки при принудительном внесении сбоев (для этого в настоящей работе подготовлен инструмент, но систематических замеров не выполнено) и нагрузочным тестированием на большом числе одновременных потоковых запросов. Во-вторых, представляет интерес исследование возможности устранения зависимости от лагающего атрибута `upper_row_index` источника очереди за счёт перехода на интерфейс `SupportsAdmissionControl` Apache Spark, что упростит реализацию источника и повысит его устойчивость к запаздыванию метаданных. В-третьих, на стороне SPYT может быть реализован альтернативный потоковый режим на основе `ContinuousExecution Spark` для сценариев, в которых строго однократная семантика обеспечивается схемой результирующей таблицы, а не транзакцией движка.

ПРИЛОЖЕНИЕ А

(Обязательное)

Листинги исходного кода ключевых компонентов реализации

В настоящем приложении приведён исходный код классов и методов адаптера SPYT, разработанных или существенно изменённых в рамках настоящей работы. Идентификаторы Scala- и Java-классов, методов и трейтов сохранены в исходной форме. Импорты и пакетные объявления опущены везде, где это не существенно для понимания логики; функционально значимые участки приведены в полном объёме.

Классы, которые не претерпели изменений в рамках настоящей работы (например, утилитные `YtQueueRange`, `YtQueueRDD`, агент байткода `SparkPatchAgent` и сопутствующие аннотации `@Decorate` и `@PatchSource`), в приложении не дублируются — в основном тексте на них даются прямые ссылки на репозиторий SPYT.

Листинг А.1 — Трейт `StreamingTransactionSupport` и трейт `StreamingTransactionHandle` (модуль `spark-adapter-api`)

```
trait StreamingTransactionHandle {
  def getId: String

  def getStickyProxyAddress: Option[String] = None

  def commit(): Unit

  def abort(): Unit
}

trait StreamingTransactionSupport {
  def isTransactionalStreamingEnabled(sparkSession: SparkSession): Boolean
}
```

```

def createTransaction(sparkSession: SparkSession):
    StreamingTransactionHandle

def setTransaction(handle: StreamingTransactionHandle): Unit = {
    setTransactionId(handle.getId)
    setStickyProxyAddress(handle.getStickyProxyAddress)
}

def setTransactionId(txId: String): Unit

def setStickyProxyAddress(address: Option[String]): Unit = {}

def clearTransactionId(): Unit

def isRecoveryNeeded: Boolean = false

def markRecoveryNeeded(): Unit = {}

def clearRecoveryNeeded(): Unit = {}
}

object StreamingTransactionSupport {
    lazy val instance: StreamingTransactionSupport =
        ServiceLoader.load(classOf[StreamingTransactionSupport]).findFirst().get()
}

```

Листинг А.2 — Реализация YtSaurusStreamingTransactionSupport и YtStreamingTransactionHandle (модуль data-source-extended)

```

class YtSaurusStreamingTransactionSupport extends
    StreamingTransactionSupport {

    override def isTransactionalStreamingEnabled(sparkSession:
        SparkSession): Boolean =
        sparkSession.ytConf(Streaming.Transactional)

    override def createTransaction(sparkSession: SparkSession):
        StreamingTransactionHandle = {
        val ytClient = YtClientProvider.ytClient(
            YtClientConfigurationConverter.ytClientConfiguration(sparkSession.sparkContext.get
            )
        )
        val transaction = YtWrapper.createTransaction(
            parent = None,
            timeout = Duration.ofMinutes(5),
            sticky = true
        )(ytClient)

        new YtStreamingTransactionHandle(transaction)
    }

    override def setTransaction(handle: StreamingTransactionHandle):
        Unit =
        YtStreamingTransactionContext.setContext(
            StreamingTxContext(handle.getId, handle.getStickyProxyAddress))

    override def setTransactionId(txId: String): Unit =
        YtStreamingTransactionContext.setTransactionId(txId)

    override def setStickyProxyAddress(address: Option[String]): Unit =
        YtStreamingTransactionContext.setStickyProxyAddress(address)
}

```

```

    override def clearTransactionId(): Unit =
        YtStreamingTransactionContext.clearTransactionId()

    override def isRecoveryNeeded: Boolean =
        YtStreamingTransactionContext.isRecoveryNeeded
    override def markRecoveryNeeded(): Unit =
        YtStreamingTransactionContext.markRecoveryNeeded()
    override def clearRecoveryNeeded(): Unit =
        YtStreamingTransactionContext.clearRecoveryNeeded()
}

class YtStreamingTransactionHandle(transaction: ApiServiceTransaction)
    extends StreamingTransactionHandle {
    private val log = LoggerFactory.getLogger(getClass)

    override def getId: String = transaction.getId.toString

    override def getStickyProxyAddress: Option[String] = {
        try {
            val classes = Iterator.iterate[Class[_]](transaction.getClass)(c
                => c.getSuperclass)
                .takeWhile(_ != null)
            val mOpt = classes.flatMap(_.getDeclaredMethods.iterator)
                .find(m => m.getName == "getRpcProxyAddress" &&
                    m.getParameterCount == 0)
            mOpt.flatMap { m =>
                m.setAccessible(true)
                Option(m.invoke(transaction)).map(_.toString)
            }
        } catch {
            case NonFatal(_) => None
        }
    }

    override def commit(): Unit = transaction.commit().join()

    override def abort(): Unit = {
        try {
            transaction.abort().join()
        } catch {
            case e: IllegalStateException =>
                log.warn("Transaction abort() failed with
                    IllegalStateException: {}", e.getMessage)
        }
    }
}

```

Листинг А.3 — YtStreamingTransactionContext (потокоразовое хранилище контекста)

```

case class StreamingTxContext(txId: String, stickyAddress:
    Option[String])

object YtStreamingTransactionContext {
    private val currentTransactionContext:
        InheritableThreadLocal[Option[StreamingTxContext]] =
        new InheritableThreadLocal[Option[StreamingTxContext]]() {
            override def initialValue(): Option[StreamingTxContext] = None
        }

    private val recoveryNeededFlag: AtomicBoolean = new
        AtomicBoolean(false)
}

```

```

def isRecoveryNeeded: Boolean = recoveryNeededFlag.get()
def markRecoveryNeeded(): Unit = recoveryNeededFlag.set(true)
def clearRecoveryNeeded(): Unit = recoveryNeededFlag.set(false)

def get: Option[StreamingTxContext] =
  currentTransactionContext.get()

def currentTransactionId: Option[String] =
  currentTransactionContext.get().map(_.txId)

def currentStickyProxyAddress: Option[String] =
  currentTransactionContext.get().flatMap(_.stickyAddress)

def setContext(ctx: StreamingTxContext): Unit =
  currentTransactionContext.set(Some(ctx))

def setTransactionId(txId: String): Unit = {
  val sticky =
    currentTransactionContext.get().flatMap(_.stickyAddress)
  currentTransactionContext.set(Some(StreamingTxContext(txId,
    sticky)))
}

def setStickyProxyAddress(address: Option[String]): Unit = {
  currentTransactionContext.get() match {
    case Some(ctx) =>
      currentTransactionContext.set(Some(ctx.copy(stickyAddress =
        address)))
    case None =>
  }
}

def clearTransactionId(): Unit = currentTransactionContext.remove()
}

```

Листинг А.4 — Декоратор MicroBatchExecutionDecorators (модуль spark-adapter/impl/spark-3.5.0)

```

@Decorate
@OriginClass("org.apache.spark.sql.execution.streaming.MicroBatchExecution")
@Applicability(from = "3.5.0")
class MicroBatchExecutionDecorators extends Logging {

  var availableOffsets: StreamProgress = ???
  var commitLog: CommitLog = ???
  var currentBatchId: Long = ???

  @DecoratedMethod
  private def runBatch(sparkSessionToRunBatch: SparkSession): Unit = {
    val sts = StreamingTransactionSupport.instance
    val transactionalStreamingEnabled =
      sts.isTransactionalStreamingEnabled(sparkSessionToRunBatch)

    if (!transactionalStreamingEnabled) {
      __runBatch(sparkSessionToRunBatch)
      return
    }

    if (sts.isRecoveryNeeded) {

```

```

        logWarning("Transactional streaming entering batch with
            recoveryNeeded flag set. " +
            "Previous batch failed and the YTsaurus consumer state may
            diverge from the Spark commit log.")
    }

    val currentTransaction: StreamingTransactionHandle =
        sts.createTransaction(sparkSessionToRunBatch)
    sts.setTransaction(currentTransaction)
    val batchIdAtEntry = currentBatchId
    var commitLogWritten = false

    try {
        __runBatch(sparkSessionToRunBatch)
        commitLogWritten = true
        MicroBatchExecutionDecorators.commitOffsets(availableOffsets)
        currentTransaction.commit()
        sts.clearRecoveryNeeded()
    } catch {
        case e: Exception =>
            sts.markRecoveryNeeded()
            try { currentTransaction.abort() } catch { case NonFatal(_) =>
                () }
            if (commitLogWritten) {

                MicroBatchExecutionDecorators.deleteCommitLogEntry(commitLog,
                    batchIdAtEntry)
            }
            throw e
    } finally {
        sts.clearTransactionId()
    }
}

private def __runBatch(sparkSessionToRunBatch: SparkSession): Unit =
    ???
}

object MicroBatchExecutionDecorators extends Logging {
    def commitOffsets(availableOffsets: StreamProgress): Unit = {
        for ((src, offset) <- availableOffsets.toList) {
            src match {
                case source: Source =>
                    source.commit(offset.asInstanceOf[Offset])
                case _ =>
            }
        }
    }
}

def deleteCommitLogEntry(commitLog: CommitLog, batchId: Long): Unit
    = {
    try {
        commitLog.purgeAfter(batchId - 1)
        logWarning(s"Rolled back commit log entry $batchId because YT
            transaction commit failed")
    } catch {
        case t: Throwable =>
            logWarning(
                s"Failed to roll back commit log entry $batchId:
                ${t.getClass.getName}: ${t.getMessage}", t)
    }
}

```

```

    }
}

```

Листинг А.5 — YtStreamingSource (источник потока, модуль data-source-extended)

```

class YtStreamingSource(sqlContext: SQLContext, consumerPath: String,
    queuePath: String,
    val schema: StructType, parameters:
    Map[String, String],
    offsetProvider: YtQueueOffsetProvider =
    YtQueueOffsetProvider)
    (implicit val yt: CompoundClient)
    extends Source with Logging with SupportsAdmissionControl {

    protected[streaming] lazy val cluster: String =
        YtWrapper.clusterName()
    private val includeServiceColumns =
        parameters.get("include_service_columns").exists(_._toBoolean)

    private var lastCommittedOffset: YtQueueOffset =
        getLastCommittedOffset
    private var maxOffset: Option[YtQueueOffset] = None

    private def transactionalEnabled: Boolean =
        Try(sqlContext.sparkSession.ytConf(Streaming.Transactionnal)).getOrElse(false)

    protected[streaming] def getMaxOffset: Option[YtQueueOffset] = {
        offsetProvider.getMaxOffset(cluster, queuePath) match {
            case Success(newMaxOffset) =>
                if (!(newMaxOffset >= lastCommittedOffset)) return None
                else {
                    if (maxOffset.isDefined && !(newMaxOffset >= maxOffset.get))
                    {
                        logWarning(s"New upper index < old upper index:
                        $newMaxOffset < ${maxOffset.get}.")
                    } else {
                        maxOffset = Some(newMaxOffset)
                    }
                }
            case Failure(exception) =>
                logWarning(s"Failed to get new max offset:
                ${exception.getMessage}", exception)
        }
        maxOffset
    }

    protected[streaming] def getLastCommittedOffset: YtQueueOffset = {
        lastCommittedOffset = offsetProvider.getCurrentOffset(cluster,
            consumerPath, queuePath)
        lastCommittedOffset
    }

    override def getDefaultReadLimit: ReadLimit =
        parameters.get("max_rows_per_partition") match {
            case Some(maxRows) =>
                ReadLimit.compositeLimit(Array(ReadLimit.maxRows(maxRows.toLong)))
            case None => ReadLimit.allAvailable()
        }

    override def latestOffset(startOffset: streaming.Offset, limit:
        ReadLimit): streaming.Offset = {
        val startOffsetParsed = if (transactionalEnabled) {

```

```

        getLastCommittedOffset
    } else {
        val cached: Option[YtQueueOffset] =
            Option(startOffset).map(YtQueueOffset.apply)
        if (cached.isEmpty) getLastCommittedOffset
        else if (cached.get >= lastCommittedOffset) cached.get
        else lastCommittedOffset
    }

    val newMaxOffsetOpt = getMaxOffset
    if (newMaxOffsetOpt.isEmpty) return null
    val maxOffset = newMaxOffsetOpt.get

    val maxRows = limit match {
        case c: CompositeReadLimit => c.getReadLimits.collectFirst {
            case r: ReadMaxRows => r.maxRows() }
        case r: ReadMaxRows => Some(r.maxRows())
        case _ => None
    }
    if (maxRows.isEmpty) return maxOffset

    val partitions = maxOffset.partitions.map { case (i, upper) =>
        val start = startOffsetParsed.partitions.getOrElse(i, -1L)
        i -> math.min(start + maxRows.get, upper)
    }
    YtQueueOffset(cluster, queuePath, partitions)
}

override def getBatch(start: Option[Offset], end: Offset): DataFrame
    = {
    val rdd = if (start.isDefined && start.get == end) {
        sqlContext.sparkContext.emptyRDD[InternalRow].setName("empty")
    } else {
        val preparedStart = if (transactionalEnabled)
            lastCommittedOffset
        else start.map(YtQueueOffset.apply).filter(_ >=
            lastCommittedOffset).getOrElse(lastCommittedOffset)
        val preparedEnd = YtQueueOffset(end)

        if (!(preparedEnd >= preparedStart)) {
            sqlContext.sparkContext.emptyRDD[InternalRow].setName("empty")
        } else {
            val ranges = YtQueueOffset.getRanges(preparedStart,
                preparedEnd)
            new YtQueueRDD(sqlContext.sparkContext, schema, consumerPath,
                queuePath,
                ranges, includeServiceColumns).setName("yt")
        }
    }
    StreamingUtils.createStreamingDataFrame(sqlContext, rdd, schema)
}

override def commit(end: Offset): Unit = {
    val txContext = YtStreamingTransactionContext.get

    if (txContext.isEmpty) {
        val transactionalEnabled =
            sqlContext.sparkSession.ytConf(Streaming.Transaction)
        if (transactionalEnabled) return
        offsetProvider.advance(consumerPath, YtQueueOffset(end),
            lastCommittedOffset, maxOffset, None)(yt)
    }
    return
}

```

```

    }

    val parentTransactionId = txContext.map(_.txId)
    val stickyAddr = txContext.flatMap(_.stickyAddress)
    val advanceClient: CompoundClient = stickyAddr match {
        case Some(addr) =>
            val conf = YtClientConfigurationConverter

                .ytClientConfiguration(sqlContext.sparkSession.sparkContext.getConf)
                YtClientProvider.ytClient(conf.copy(fixedProxyAddress =
                    Some(addr)))
        case None => yt
    }

    offsetProvider.advance(consumerPath, YtQueueOffset(end),
        lastCommittedOffset, maxOffset,
        parentTransactionId)(advanceClient)
}

override def stop(): Unit = logDebug("Close YtStreamingSource")
}

```

Листинг А.6 — YtStreamingSink (приёмник потока, модуль data-source-extended)

```

class YtStreamingSink(sqlContext: SQLContext, queuePath: String,
    parameters: Map[String, String]) extends Sink
    with Logging {
    @volatile private var latestBatchId = -1L

    override def toString: String = "YtStreamingSink"

    override def addBatch(batchId: Long, data: DataFrame): Unit = {
        if (batchId <= latestBatchId) {
            logInfo(s"Skipping already committed batch $batchId")
        } else {
            val sparkContext = sqlContext.sparkSession.sparkContext

            val ytClientConfiguration =
                YtClientConfigurationConverter.ytClientConfiguration(sparkContext.getConf)
            val bcYtClientConfiguration =
                sparkContext.broadcast(ytClientConfiguration)

            val wConfig = SparkYtWriteConfiguration(sqlContext)
            val bcWriterConfig = sparkContext.broadcast(wConfig)

            val path = YPathEnriched.fromPath(new Path(queuePath))
            val bcPath = sparkContext.broadcast(path)
            val bcParameters = sparkContext.broadcast(parameters)
            val bcSchema = sparkContext.broadcast(data.schema)

            val txContext = YtStreamingTransactionContext.get
            val bcParentTransactionId = txContext.map(ctx =>
                sparkContext.broadcast(ctx.txId))
            val bcStickyProxyAddress =
                txContext.flatMap(_.stickyAddress).map(sparkContext.broadcast)

            data.queryExecution.toRdd.foreachPartition { partitionIterator
                =>
                    if (partitionIterator.hasNext) {
                        val txId = bcParentTransactionId.map(_.value)
                        val stickyAddr = bcStickyProxyAddress.map(_.value)

```



```

    implicit val partitionYtClient: CompoundClient = stickyAddr
  match {
    case Some(addr) =>
      YtClientProvider.ytClient(
        bcYtClientConfiguration.value.copy(fixedProxyAddress =
Some(addr)))
    case None =>
      YtClientProvider.ytClient(bcYtClientConfiguration.value)
  }
  val attachedTx = txId.map(id =>
YtWrapper.attachTransaction(id))
  val dynamicTableWriter = new YtDynamicTableWriter(
    bcPath.value, bcSchema.value, bcWriterConfig.value,
    bcParameters.value, attachedTx)
  try {
    partitionIterator.foreach { row =>
dynamicTableWriter.write(row) }
  } finally {
    dynamicTableWriter.close()
  }
}
}
latestBatchId = batchId
}
}
}
}

```

Листинг А.7 — YtDynamicTableWriter (приёмник записи в динамическую таблицу, модуль data-source)

```

class YtDynamicTableWriter(richPath: YPathEnriched, schema:
  StructType,
                        wConfig: SparkYtWriteConfiguration,
  options: Map[String, String],
                        parentTransaction:
  Option[ApiServiceTransaction] = None)
  (implicit ytClient: CompoundClient) extends
  OutputWriter {

  override val path: String = richPath.toStringPath
  private val writeSchemaConverter = WriteSchemaConverter(options)
  private val typeV3: Boolean = writeSchemaConverter.typeV3Format
  private val tableSchema: TableSchema =
    TableSchema.fromYTree(YtWrapper.attribute(path, "schema"))
  private val rowConverter: DynTableRowConverter =
    new DynTableRowConverter(schema, tableSchema, typeV3)
  private var count = 0
  private var modifyRowsRequestBuilder: ModifyRowsRequest.Builder = _

  initialize()

  def write(row: Seq[Any]): Unit = {
    val preparedRow = rowConverter.convertRow(row)
    modifyRowsRequestBuilder.addInsert(preparedRow.asJava)
    count += 1
    if (count == wConfig.dynBatchSize) commitBatch()
  }

  override def write(row: InternalRow): Unit =
    write(row.toSeq(schema))

  override def close(): Unit = {

```

```

    if (count > 0) commitBatch()
  }

  private def initBatch(): Unit = {
    modifyRowsRequestBuilder = ModifyRowsRequest.builder()
      .setPath(richPath.toStringYPath).setSchema(tableSchema)
    count = 0
  }

  private def commitBatch(): Unit = {
    YtMetricsRegister.time(writeBatchTime, writeBatchTimeSum) {
      val request = modifyRowsRequestBuilder.build()
      YtWrapper.insertRows(request, parentTransaction)
    }
    initBatch()
  }

  private def initialize(): Unit = {
    initBatch()
    YtMetricsRegister.register()
  }
}

```

Листинг А.8 — YtTransactionUtils: создание, прикрепление и пинг транзакций (модуль yt-wrapper)

```

trait YtTransactionUtils { self: LogLazy =>

  def createTransaction(timeout: Duration, sticky: Boolean)
    (implicit yt: CompoundClient):
    ApiServiceTransaction =
      createTransaction(None, timeout, sticky)

  def createTransaction(parent: Option[String], timeout: Duration,
    sticky: Boolean = false, title: Option[String]
    = None,
    pingPeriod: Duration = Duration.ofSeconds(30))
    (implicit yt: CompoundClient):
    ApiServiceTransaction = {
    val request = new StartTransaction(
      if (sticky) TransactionType.Tablet else
      TransactionType.Master).toBuilder
      .setTransactionTimeout(timeout)
      .setTimeout(timeout)
      .setPing(true)
      .setPingPeriod(pingPeriod)
    title.foreach(t => request.setAttributes(
      Collections.singletonMap("title", YTree.stringNode(t))))
    parent.foreach(p => request.setParentId(GUID.valueOf(p)))
    yt.startTransaction(request.build()).join()
  }

  def attachTransaction(transactionId: String, ping: Boolean = false,
    pingPeriod: Option[Duration] = None)
    (implicit yt: CompoundClient):
    ApiServiceTransaction = {
    val builder = AttachTransaction.builder()
      .setTransactionId(GUID.valueOf(transactionId))
      .setPing(ping)
    pingPeriod.foreach(builder.setPingPeriod)
    yt.attachTransaction(builder.build()).join()
  }
}

```

```

}

def abortTransaction(guid: String)(implicit yt: CompoundClient):
  Unit =
    yt.abortTransaction(GUID.valueOf(guid)).join()

def commitTransaction(guid: String)(implicit yt: CompoundClient):
  Unit =
    yt.commitTransaction(GUID.valueOf(guid)).join()

def pingTransaction(tr: ApiServiceTransaction, interval: Duration)
  (implicit yt: CompoundClient, ec:
    ExecutionContext): Cancellable[Unit] = {
  @tailrec
  def ping(cancel: Future[Unit], retry: Int): Boolean = {
    try {
      if (!cancel.isCompleted) { tr.ping().join(); true } else false
    } catch {
      case e: Throwable =>
        if (retry > 0) {
          Thread.sleep(new Random().nextInt(2000) + 100)
          ping(cancel, retry - 1)
        } else false
    }
  }
  cancellable { cancel =>
    var success = true
    while (!cancel.isCompleted && success) {
      success = ping(cancel, 3)
      Thread.sleep(interval.toMillis)
    }
  }
}

```

Листинг А.9 — Методы записи строк `YtDynTableUtils.insertRows` (модуль `yt-wrapper`)

```

trait YtDynTableUtils { self: YtCypressUtils =>

  def insertRows(path: String, schema: TableSchema, rows:
    Seq[Seq[Any]],
    parentTransaction: Option[ApiServiceTransaction] =
      None)
    (implicit yt: CompoundClient): Unit = {
    processModifyRowsRequest(
      ModifyRowsRequest.builder()
        .setPath(formatPath(path))
        .setSchema(schema)
        .addInserts(rows.map(_._.asJava).asJava)
        .build(),
      parentTransaction)
  }

  def insertRows(modifyRowsRequest: ModifyRowsRequest,
    parentTransaction: Option[ApiServiceTransaction])
    (implicit yt: CompoundClient): Unit =
    processModifyRowsRequest(modifyRowsRequest, parentTransaction)

  def insertRows(modifyRowsRequest: ModifyRowsRequest,
    parentTransactionId: String)

```

```

        (implicit yt: CompoundClient): Unit =
yt.modifyRows(GUID.valueOf(parentTransactionId),
    modifyRowsRequest).join()

private def processModifyRowsRequest(request: ModifyRowsRequest,
    transaction:
    Option[ApiServiceTransaction] = None)
    (implicit yt: CompoundClient):
    Unit = {
val f: ApiServiceTransaction => Unit =
    _.modifyRows(request).get(1, TimeUnit.MINUTES)
runWithDefinedTxOrRetry(f, transaction)
}

def runWithDefinedTxOrRetry[T](f: ApiServiceTransaction => T,
    tx: Option[ApiServiceTransaction] =
    None,
    attemptLimit: Int = 3)
    (implicit yt: CompoundClient): T = tx
    match {
case Some(tx) => f(tx)
case None => runWithRetry(f, attemptLimit)
}
}

```

Листинг А.10 — YtQueueUtils: продвижение consumer-offset очереди (модуль yt-wrapper)

```

trait YtQueueUtils { self: YtCypressUtils with YtTransactionUtils =>

def pullConsumer(consumerPath: String, queuePath: String,
    partitionIndex: Int,
    offset: Long, maxRowCount: Long)
    (implicit yt: CompoundClient): QueueRowset = {
val options = RowBatchReadOptions.builder()
    .setMaxRowCount(maxRowCount)
    .setMaxDataWeight(DataSize.fromTeraBytes(1)).build()
val request = PullConsumer.builder()
    .setConsumerPath(YPath.simple(consumerPath))
    .setQueuePath(YPath.simple(queuePath))
    .setPartitionIndex(partitionIndex)
    .setOffset(offset)
    .setRowBatchReadOptions(options)
    .build()
runWithRetry(() => yt.pullConsumer(request).get(30000,
    TimeUnit.MILLISECONDS))
}

def advanceConsumer(consumerPath: YPath, queuePath: YPath,
    partitionIndex: Int,
    newOffset: Long, transaction:
    ApiServiceTransaction): Unit = {
val request = AdvanceConsumer.builder()
    .setConsumerPath(consumerPath)
    .setQueuePath(queuePath)
    .setPartitionIndex(partitionIndex)
    .setNewOffset(newOffset)
    .build()
transaction.advanceConsumer(request).join()
}
}

```

```

def advanceConsumer(consumerPath: YPath, queuePath: YPath,
    partitionIndex: Int,
    newOffset: Long, parentTransactionId: String)
    (implicit yt: CompoundClient): Unit = {
    val request = AdvanceConsumer.builder()
        .setConsumerPath(consumerPath)
        .setQueuePath(queuePath)
        .setPartitionIndex(partitionIndex)
        .setNewOffset(newOffset)
        .setTransactionId(GUID.valueOf(parentTransactionId))
        .build()
    yt.advanceConsumer(request).join()
}

def registerQueueConsumer(consumerPath: YPath, queuePath: YPath,
    vital: Boolean = true)
    (implicit yt: CompoundClient): Unit = {
    val request = RegisterQueueConsumer.builder()
        .setConsumerPath(consumerPath)
        .setQueuePath(queuePath)
        .setVital(vital)
        .build()
    yt.registerQueueConsumer(request).join()
}
}

```

Листинг А.11 — Конфигурационная запись `SparkYtConfiguration.Streaming` (модуль `data-source`)

```

object Streaming {
    case object Transactional extends ConfigEntry[Boolean]
        ("streaming.transactional", Some(false))
}

```

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Akidau T., Bradshaw R., Chambers C. et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing // Proceedings of the VLDB Endowment. — 2015. — Vol. 8, № 12. — P. 1792–1803. (Scopus)
2. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol: O'Reilly Media, 2017. — 590 p.
3. Helland P. Idempotence Is Not a Medical Condition // Communications of the ACM. — 2012. — Vol. 55, № 5. — P. 56–65. (Scopus)
4. Wang G., Chen L., Dikshit A. et al. Consistency and Completeness: Rethinking Distributed Stream Processing in Apache Kafka // Proceedings of the 2021 International Conference on Management of Data (SIGMOD). — 2021. — P. 2602–2613. (Scopus)
5. Armbrust M., Das T., Torres J. et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark // Proceedings of the 2018 ACM International Conference on Management of Data (SIGMOD). — 2018. — P. 601–613. (Scopus)

6. Документация YTsaurus [Электронный ресурс]. — Режим доступа: <https://ytsaurus.tech/docs/ru/> (дата обращения: 15.05.2026).
7. Zhou J., Xu M., Shraer A. et al. FoundationDB: A Distributed Unbundled Transactional Key Value Store // Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21). — New York: ACM, 2021. — P. 2653–2666. — DOI: 10.1145/3448016.3457559. (Scopus)
8. Budiu M., Chajed T., McSherry F., Ryzhyk L., Tannen V. DBSP: Automatic Incremental View Maintenance for Rich Query Languages // Proceedings of the VLDB Endowment. — 2023. — Vol. 16, № 7. — P. 1601–1614. (Scopus)
9. Wang Y., Liu Z. A Sneak Peek at RisingWave: A Cloud-Native Streaming Database // Proceedings of the 16th ACM International Conference on Distributed and Event-based Systems (DEBS '22). — New York: ACM, 2022. — P. 175–176. (Scopus)
10. Schulze R., Schreiber T., Yatsishin I., Dahimene R., Milovidov A. ClickHouse — Lightning Fast Analytics for Everyone // Proceedings of the VLDB Endowment. — 2024. — Vol. 17, № 12. — P. 3731–3744. — DOI: 10.14778/3685800.3685802. (Scopus)
11. Silvestre P.F., Fragkoulis M., Spinellis D., Katsifodimos A. Clonos: Consistent Causal Recovery for Highly-Available Streaming Dataflows // Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21). — New York: ACM, 2021. — P. 1637–1650. — DOI: 10.1145/3448016.3457320. (Scopus)
12. Mei Y., Li Y., Hu Z. et al. Disaggregated State Management in Apache Flink 2.0 // Proceedings of the VLDB Endowment. — 2025. — Vol. 18, № 12. — P. 4846–4859. (Scopus)
13. Fragkoulis M., Carbone P., Kalavri V., Katsifodimos A. A Survey on the Evolution of Stream Processing Systems // The VLDB Journal. — 2024. — Vol. 33, № 2. — P. 507–541. — DOI: 10.1007/s00778-023-00819-8. (WoS/Scopus)
14. Кузнецов С.Д., Великов П.Е., Фу Ц. Аналитика в реальном времени: преимущества, ограничения и компромиссы // Программирование. — 2023. — № 1. — С. 3–31. — DOI: 10.31857/S0132347423010053. (БАК)
15. Сазонтьев В.В., Ступников С.А., Захаров В.Н. Расширяемый подход к слиянию данных в распределённых вычислительных средах // Информатика и её применения. — 2023. — Т. 17, № 4. — С. 42–47. (БАК)
16. Зорин А.А., Чернецкая И.Е. Подходы к развёртыванию в распределённой обработке сложных событий // Труды Института системного программирования РАН. — 2023. — Т. 35, № 3. — [Электронный ресурс]. — Режим доступа: <https://ispranproceedings.elpub.ru/jour/article/view/1656> (дата обращения: 15.05.2026). (БАК)
17. Воробьёва Г.Р., Фарваев Э.Ф. Архитектура распределённой системы сбора и обработки геопространственных данных на основе паттернов веб-проектирования // Программная инженерия. — 2023. — Т. 14, № 10. — С. 482–492. (БАК)
18. Коробейников А.Г. Применение методов Big Data для предобработки данных из сети INTERMAGNET // Известия высших учебных заведений. Приборостроение. — 2023. — Т. 66, № 7. — С. 533–538. — DOI: 10.17586/0021-3454-2023-66-7-533-538. (БАК)
19. Бурдонов И.Б., Евтушенко Н.В., Косачев А.С. Реализация распределённых и параллельных вычислений в сети SDN // Труды Института системного программирования РАН. — 2022. — Т. 34, № 3. — [Электронный ресурс]. — Режим доступа: <https://www.mathnet.ru/php/contents.phtml?jrmid=tisp&volume=34&issue=3&year=2022> (дата обращения: 15.05.2026). (БАК)

20. Apache Spark Structured Streaming Programming Guide [Электронный ресурс]. — Режим доступа: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (дата обращения: 15.05.2026). (техдокументация)
21. Apache Flink Documentation [Электронный ресурс]. — Режим доступа: <https://flink.apache.org/> (дата обращения: 15.05.2026). (техдокументация)
22. Apache Kafka Documentation: Design — Message Delivery Semantics and Transactions [Электронный ресурс]. — Режим доступа: <https://kafka.apache.org/documentation/#semantics> (дата обращения: 15.05.2026). (техдокументация)
23. SPYT (Spark over YTsauros) Documentation [Электронный ресурс]. — Режим доступа: <https://ytsaurus.tech/docs/en/user-guide/data-processing/spyt/overview> (дата обращения: 15.05.2026). (техдокументация)
24. YTsauros Source Code Repository [Электронный ресурс]. — Режим доступа: <https://github.com/ytsaurus/ytsaurus> (дата обращения: 15.05.2026). (техдокументация)